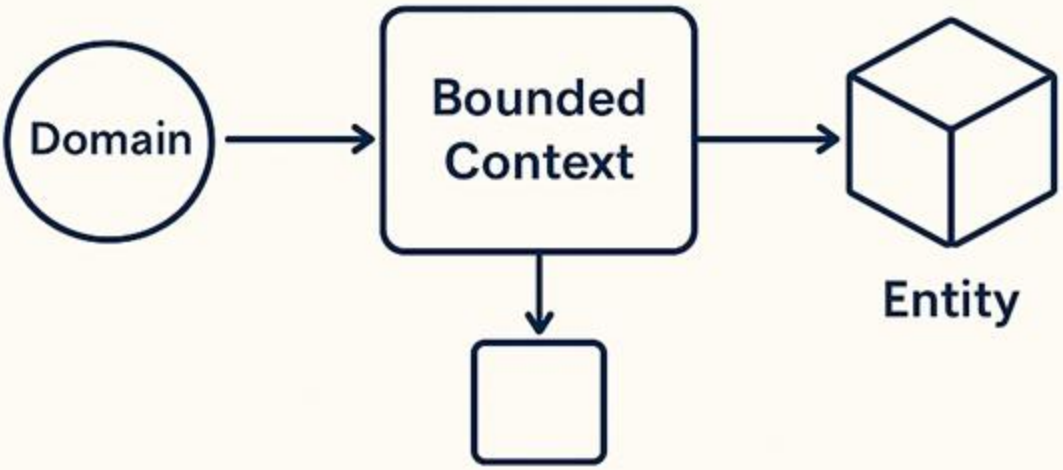
DDD

? ? ?

UNDERSTANDING DOMAIN-DRIVEN DESIGN



MAHDI KAZEMI

فهرست

DDD چیست؟ 4

مفاهیمی که DDD به آنها مکررا اشاره کرده 4

Domain 4

Core Domain 4

Model 4

Domain Model 5

Ubiquitous Language 5

Communication and Collaboration 5

Iterative Development and Refactoring 7

موارد Implicit و Explicit 8

فرآیند طراحی استراتژیک 10

فرآیند مدل سازی 13

تعریف Subdomain 13

Core Subdomain 13

Supporting Subdomain 14

Generic Subdomain 14

ارتباط Bounded Context و Subdomain ها 15

تعریف Bounded Context 15

تعریف Shared Kernel 17

تعریف Customer/Supplier 18

تعریف Conformist 20

تعریف Anti-Corruption Layer 21

تعریف Separate Ways 25

تعریف Open Host Service 26

تعریف Partnership 27

تفاوت Shared Kernel با Partnership 28

تعریف Context Mapping 29

سند چشم انداز دامنه 31

مستند سازی دامنه 32

جداسازی دامنه 33

مکانیزم ها در Sub Domain ها..... 33
Event Storming..... 36
تفاوت Event Storming و Domain Story Telling..... 36
Example Mapping..... 37
Core Domain Rafactoring..... 38
فرآیند طراحی فنی..... 39
فرآیند پیاده سازی Model در نرم افزار..... 41
Entity..... 41
Value Object..... 42
Service..... 43
Modules..... 43
Aggregates..... 44
Factories..... 45
Repository..... 46
جمع بندی..... 48
منابع..... 50

DDD چیست؟

Domain Driven Design یک رویکرد در طراحی و توسعه نرم افزار است. تمرکز این رویکرد بر **مدل سازی** فضای مساله است. معمولا از این رویکرد برای طراحی و توسعه نرم افزار های استفاده می‌شود که فضای مساله آنها شامل قوانین بیزینسی پیچیده است و اصلا مناسب بیزینس با قوانین ساده نیست، به فرض مثال برای طراحی نرم افزار معاملات بورسی استفاده از DDD یکی از بهترین گزینه هاست اما برای توسعه نرم افزار بلاگ یه کلینیک پزشکی احتمالا گزینه مناسبی نیست.

مفاهیمی که DDD به آنها مکررا اشاره کرده

Domain: هر نرم افزاری ساخته می‌شود تا در حوزه خاصی خدمت رسانی کند و مشکلی را حل کند، این مشکل یا حوزه خاص، فضای مساله یا Domain نامیده می‌شود. Domain هر نرم افزاری دلیل وجود آن نرم افزار است.

اجزای تکیل دهنده Domian شامل قوانین، اصطلاحات، فرآیندها و رفتارهای مربوط به حوزه ای است که نرم افزار برای حل یک مشکل خاص در آن حوزه ساخته شده است. به فرض مثال Domain یک نرم افزار فروشگاه اینترنتی شامل: فرآیند سفارش محصول، مشتری، ثبت نام مشتری، نحوه پرداخت، نحوه ارسال محصول و ... است.

Core Domain: فضای مساله معمولا آن قدر بزرگ است که داخل آن می‌توان مسائل مختلف را پیدا کرد اما در هر فضای مساله ای یک مساله مهمترین مساله موجود است، مساله ای که یک شرکت راه حلی برای آن مساله مهم دارد که مزیت رقابتی آن شرکت با رقبا است اصلا دلیل وجود پروژه نرم افزاری است که، شکل گرفته یا می‌خواهد شکل بگیرد. به این اصلی ترین مساله اصطلاحا Core Domain گفته می‌شود. به توصیه DDD باید بیشترین هزینه ای که در پروژه نرم افزاری می‌شود برای Core Domain باشد.

Model: یک Model تجسمی از واقعیت است اما نه تمام واقعیت. در واقع هر Model بخشی از دنیای واقعی را در بر می‌گیرد که مرتبط به یک مساله خاص است. خاصیت برجسته یک Model ساده سازی آن از یک مساله پیچیده است یعنی وقتی یک مساله ای را از دنیای واقعی به شکل Model تعریف می‌کنید بخش هایی از واقعیت را داخل Model درگیر می‌کنید که ضمن حل مشکل، آن مشکل را خیلی ساده داخل پروژه نرم افزاری تعریف می‌کند.

Eric Evans فرآیند Model سازی را به فیلم سازی تشبیه می‌کند. او می‌گوید وقتی یک کارگردان یک فیلمی را می‌سازد فقط بخش هایی را در فیلم نمایش می‌دهد که مربوط به موضوع فیلم است و از نمایش دادن تمام جزییات اضافی پرهیز می‌کند. برای درک بهتر این تشبیه، فیلم سینمایی را تصور کنید که یک اتفاق سه روزه را روایت می‌کند آیا مدت زمان آن فیلم هم 3 روز است؟ خیر. آن فیلم نهایت 2 ساعت است زیرا از نمایش تمام جزییات اتفاق پرهیز کرده. Model سازی هم به همین شکل است اگر بخواهید تمام جزییات مربوط به یک مساله را Model سازی کنید احتمالا هیچ گاه موفق نشوید اگر هم موفق بشوید آن Model احتمالا آن قدر درگیر جزییات شده که نمی‌تواند مساله اصلی را به خوبی به نمایش بگذارد. پس فقط قسمت هایی از مساله اصلی را داخل Model درگیر کنید که به حل مساله کمک می‌کند.

برای Model سازی بهتر می‌توانید از ابزاری مثل دیاگرام ها استفاده کنید. انواع دیاگرام های استاندارد مثل UML دیاگرام ها و حتی دیاگرام های غیر استاندارد مثل یک دیاگرام از ارتباط Modelهای مختلف که در جلسه تحلیل تیمی روی تخته White Board کشیده شده است می‌تواند در فرآیند Model سازی به شما کمک کند.

Domain Model: همانطور که در قسمت های قبلی گفته شد، Domain همان فضای مساله است و داخل این مساله چندین مساله مرتبط به یکدیگر وجود دارند و شما نیاز دارید که تمام این مسائل را تبدیل به Model هایی کنید تا بتوانید نرم افزار مورد نیاز را شکل دهید. به تمام اجزایی که داخل Domain هستند و برای آنها Model سازی می‌شود Domain Model گفت می‌شود.

Ubiquitous Language: یک زبان مشترک بین متخصصان تکنولوژی و متخصصان بیزینس. یکی از اهداف DDD در هر پروژه نرم افزاری ایجاد یک زبان مشترک بین ضینفعان پروژه (برنامه نویسان، تحلیگران، سهام داران، مشتریان و ...) است. مزیت ایجاد این زبان مشترک کاهش ابهام ها و سوءتفاهم ها در پروژه نرم افزاری، تسریع در روند توسعه پروژه در طولانی مدت، ارتباط موثرتر بین ضینفعان پروژه است. نکته‌ی مهمی که DDD در مورد زبان مشترک عنوان می‌کند این است که به وجود آمدن و تکامل آن زمان زیادی نیاز دارد. توصیه های DDD برای پرورش و توسعه زبان مشترک:

- از Model طراحی شده به عنوان ستون فقرات ساخت زبان مشترک استفاده کنید
- هر تغییر در زبان مشترک به معنی تغییر در Model و کد برنامه نویس می‌باشد
- سعی کنید ارتباط بین افراد فنی و بیزینسی را بیشتر کنید
- سعی کنید برای هر لغتی که در تعاملات تیم فنی و تیم بیزینسی استفاده می‌کنید تعریف مشخصی داشته باشید
- برای یک مفهوم از دو لغت متفاوت استفاده نکنید
- برای پیدا کردن بهترین لغت و بهترین تعاریف سعی کنید از لغات و تعاریف جایگزین هم استفاده کنید سپس بهترین لغت و بهترین تعریف برای پرورش زبان متشرک به کار بگیرید
- سعی کنید از زبان مشترک در هر مکالمه ای که مربوط به پروژه است استفاده کنید
- از زبان مشترک در کد استفاده کنید، برای انتخاب نام متغییر ها، کلاس ها، متد ها سعی کنید از لغاتی که در زبان مشترک هستند استفاده کنید
- در تمام داکیومنت های مربوط به پروژه اعم از داکیومنت های فنی و بیزینسی از زبان مشترک استفاده کنید

Ubiquitous Language و Model و Code سه ضلع یک مثلث در DDD هستند که همیشه باید حواستان باشد در طول عمر پروژه این سه ضلع کنار هم حفظ شوند. اگر به هر دلیلی هر کدام از این سه ضلع دچار تغییر شوند نیاز به بازبینی دو ضلع دیگر برای اعمال تغییر در صورت نیاز است.

به یاد داشته باشید که برنامه نویسان و افراد فنی پروژه یک سری دایره لغات مخصوص به تخصص خود دارند و افراد متخصص کسب و کار هم یک سری دایره لغات مخصوص به کسب و کار دارند که هیچ ارتباطی به پروژه ندارد نکته مهم این است که این دایره لغات تخصصی فنی و بیزینسی هیچ جایی در زبان متشرک ندارند و تنها لغات، اصطلاحات، تعاریف و مفاهیمی که افراد بیزینسی و فنی هر دو باهم نیاز دارند بدانند تا بتوانند پروژه را توسعه دهند باید در زبان متشرک به کار گرفته شود.

Communication and Collaboration: ارتباط و تعامل درون تیمی تاثیر بسزایی در موفقیت یک پروژه نرم افزاری دارد، از همین رو در کتاب DDD به این موضوع به دفعات اشاره شده است. فرآیند شکل گیری یک پروژه نرم افزاری بدین شکل است که یک نیازمندی از سمت صاحب کسب و کار به تیم بیزینس اعلام می‌شود سپس تحلیل گران و متخصصان کسب و کار آن نیازمندی را بررسی و تحلیل می‌کنند و یک سری داکیومنت آماده می‌کنند سپس آن داکیومنت در اختیار تیم توسعه قرار می‌گیرد و نرم افزار مورد نظر شکل می‌گیرد. اما چالش این فرآیند کجاست؟ چالش ها:

- عدم قطعیت صاحب کسب و کار از چیزی که درخواست داده است
- عدم انتقال مناسب خواسته ها بین افراد
- تعامل کم بین افراد تیم
- مقاومت افراد نسبت به تغییرات پروژه
- عدم آگاهی تمام اعضای تیم نسبت به فضای مساله
- علاقه اعضای فنی تیم به کارهای فنی و تکنولوژی و عدم علاقه آنها به بیزینس و تحلیل

در بیشتر مواقع درخواستی که از سمت صاحبان کسب و کار اعلام نیاز می‌شود به اندازه کافی پخته و کامل نیست و خیلی طبیعی است که بعدها این خواسته دچار تغییر و تحول شود. فارغ از بحث بالغ بودن درخواست صاحبان کسب و کار، قدرت درک افراد و قدرت انتقال آنها از درخواست بسیار مهم است. فرض کنید یک درخواستی از سمت صاحب کسب و کار رسیده که در حال حاضر 60 درصد این نیازمندی پخته و کامل است و هنوز 40 درصد باقی شفاف نیست، حال اگر تحلیلگر تنها 40 درصد از آن 60 را درک کند و باقی آن را متوجه نشود و هنگام توضیح این درخواست به تیم توسعه برای پیاده سازی نتواند به صورت کامل آن چیزی را که متوجه شده توضیح بدهد در نهایت نتیجه این است که تیم فنی ممکن است به اندازه 10 درصد از نیازمندی صاحب کسب و کار را درک کرده باشد پس در نتیجه چیزی که در نهایت توسط تیم توسعه پیاده سازی می‌شود با آنچه که صاحب کسب و کار درخواست داده تفاوت بسیار زیادی دارد. بنابراین تحلیلگری که نیازمندی صاحب کسب و کار را بررسی می‌کند باید به خوبی و کامل نیازمندی اعلام شده را درک کند و تمام جوانب آن را بسنجد، اگر در جایی تناقصی دید با صاحب کسب و کار صحبت کند و اگر توانست به پخته تر شدن درخواست اعلامی کمک کند سپس این درخواست را به تیم فنی منتقل کند.

اما حتی اگر تحلیل باعث پخته شدن درخواست شود و نیازمندی که به تیم فنی اعلام می‌کند کامل تر باشد باز هم بسیار محتمل است که این درخواست تغییر کند و تغییر در نیازمندی ها یعنی تغییر در کد پیاده سازی شده یعنی کار بیشتر برای افراد فنی تیم پس طبیعی است که افراد فنی در برابر تغییراتی که از سمت صاحبان کسب و کار اعلام نیازمندی می‌شود مقاومت داشته باشند. برای کمتر کردن این مقاومت، تحلیگر کسب و کار باید نیازمندی هایی که از طرف صاحبان کسب و کار اعلام می‌شود را به خوبی بررسی کند و اگر می‌تواند به کامل تر شدن درخواست کمک کند تا بعدا این نیازمندی کمتر دچار تغییر و تحول شود.

یکی از مواردی که تغییر را سخت می‌کند، تغییر در چیزی است که آگاهی کمی نسبت به آن داریم. این موضوع در مورد پروژه های نرم افزاری هم صدق می‌کند یعنی یک توسعه دهنده، نرم افزاری را که نسبت به فضای مساله آن آگاه تر است راحت تر می‌تواند تغییر دهد. بنابراین هر چقدر آگاهی اعضای تیم نسبت به فضای مساله بیشتر باشد، تغییرات در زمان کمتر و با آگاهی بیشتر صورت می‌پذیرد. همچنین تغییر در یک قسمت نرم افزار باقی قسمت های نرم افزار را هم تحت تاثیر قرار می‌دهد و توسعه دهنده ای که تغییر را اعمال می‌کند اگر نداند این تغییر چه قسمت هایی را تحت تاثیر قرار می‌دهد ممکن است باعث به وجود آمدن یک فاجعه شود پس هر چقدر آگاهی اعضای تیم نسبت به فضای مساله پروژه نرم افزاری بیشتر باشد تغییرات کم هزینه تر خواهند بود.

معمولا اعضای فنی تیم علاقه زیادی به تکنولوژی و کدنویسی دارند و علاقه کمی نسبت به مسائل مربوط به کسب و کار دارند. از همین رو معمولا توسعه دهندگان علاقه دارند کدهایی که بیشتر درگیر تکنولوژی هستند را توسعه دهند و کارهای مربوط به تحلیل فضای مساله را به تحلیل گران واگذار کنند. همین موضوع باعث فاصله گرفتن توسعه دهندگان از فضای مساله و فضای کسب و کار می‌شود که نتیجه آن این است که نرم

افزار پیاده سازی شده با نیازمندی های اعلام شده از سمت صاحبان کسب و کار تفاوت های زیادی دارد و نرم افزار پیاده سازی شده آن چیزی نیست که صاحبان کسب و کار درخواست داده اند.

راه حل تمام این چالش ها ارتباط و تعامل مداوم بین اعضای تیم فنی و تیم تحلیل گران و صاحبان کسب و کار است. هر چقدر این تعاملات بیشتر باشد نیازمندی های اعلام شده بیشتر بررسی می‌شوند و قوانین کسب و کار بین افراد بیشتر مورد گفتوگو قرار میگیرند و همه این ها باعث آگاهی بیشتر تمام افراد تیم نسبت به فضای مساله و فضای کسب و کار می‌شود و هرچقدر این آگاهی بیشتر باشد نرم افزاری که توسط تیم فنی پیاده سازی می‌شود نزدیک به خواسته ی صاحب کسب و کار است و همین موضوع باعث موفق شدن یک پروژه نرم افزاری می‌شود.

Iterative Development and Refactoring: Refactoring به معنی باز طراحی نرم افزار یا بخشی از نرم افزار به طوری که عملکرد آن تغییر نکند است. هر تغییر، بدون تغییر در عملکرد موجود، طراحی را انعطاف‌پذیرتر یا قابل فهم‌تر می‌کند.

بیشتر اوقات نمی‌توان برای شروع یک پروژه نرم افزاری نسبت به تمام فضای مساله آگاه بود و بعد شروع به پیاده سازی نرم افزار کرد. شما در ابتدای پروژه نرم افزاری چیزهای کمی در مورد Domain می‌دانید و سطح پایینی از دانش مربوط به فضای مساله دارید و باید با همین دانش کم پروژه نرم افزاری را شروع به پیاده سازی کنید.

اگر قرار است با سطح دانش کم پروژه نرم افزاری را شروع کنید پس چگونه نرم افزار تکمیل می‌شود؟
پیاده سازی نرم افزار یک چرخه است که از نقطه اعلام نیازمندی از سمت صاحب کسب و کار شروع می‌شود و در نقطه نرم افزار پیاده سازی شده پایان می‌یابد، اما نقطه پایان به معنی اتمام کار نیست چون این یک چرخه است، نقطه پایان برای این است که بدانید شما یک دور از توسعه نرم افزار را تمام کرده اید و نوبت شروع دور بعد است و آنقدر این چرخه را تکرار می‌کنید تا نرم افزار به بلوغ برسد و نزدیک ترین نسخه به آن چیزی که صاحب کسب و کار درخواست داده باشد.

آیا هرباری که دانش جدیدی نسبت به فضای مساله به دست می‌آوریم باید شروع به Refactoring کنیم؟ خیر

وقتی یک نیازمندی از سمت صاحب کسب و کار اعلام می‌شود تیم تحلیل آن را تحلیل می‌کنند و دانش اولیه را پرورش می‌دهند سپس با کمک تیم فنی دانش به دست آمده را Model سازی می‌کنند سپس تیم فنی Model به دست آمده را به صورت کد در پروژه پیاده سازی می‌کند. پس کد نمایشی از Model و دانش مربوط به فضای مساله است. شما در دو شرایط باید پروژه را Refactor کنید:

- شرایطی که می‌توانید طوری کد را Refactor کنید که نمایش بهتری از Model باشد
- شرایطی که بینش عمیق تری نسبت به فضای مساله به دست آورده اید و می‌توانید Model را طوری تغییر دهید که Domain را کامل تر توصیف کند

پس هربار که دانش جدیدی نسبت به فضای مساله به دست می‌آوریم فقط در صورتی که یکی از دو شرط بالا برقرار باشد می‌توانیم پروژه را Refactor کنیم.

تعامل مداوم اعضای تیم باعث به وجود آمدن بینش عمیق نسبت به Domain در اعضای تیم می‌شود و بینش عمیق باعث تغییر Model در راستای توصیف کامل تر Domain می‌شود و برای نمایش بهتر Model در کد فرایند Refactoring شروع می‌شود.

موارد Implicit و Explicit:

یکی از مواردی که از مانع بینش عمیق افراد تیم نسبت به Domain می‌شود موارد غیرصریح(Implicit) در پروژه است. یکی از مواردی که باید تلاش زیادی برای آن صورت بگیرد تبدیل موارد غیر صریح به موارد صریح(Explicit) در پروژه است.

موارد غیرصریح(Implicit) مواردی هستند که از وجود آنها باخبر هستیم اما به صورت شفاف در کد و Model قرار ندارند. این موارد باید شناسایی شوند و به صورت مستقل در زبان مشترک، کد و Model به کار گرفته شوند.

مثال: فرض کنید در پروژه یک فروشگاه اینترنتی میخواهیم قانونی داشته باشیم که اگر هر مشتری سبد خریدی با بیش از مبلغ 100 دلار داشت کل سبد خرید شامل 10 درصد تخفیف شود. این مثال را با دو مدل Implicit و Explicit میخواهیم پیاده سازی کنیم:

اگر Implicit باشد:

```
public decimal CalculateCartPrice(List<Item> items){
    //some code
    //some code
    //some code
    var sumPrice = items.sum(p => p.Price);
    if(sumPrice >= 100)
        return sumPrice - sumPrice/10;
    else
        return sumPrice;
}
```

اگر Explicit باشد:

```
public decimal CalculateCartPrice(List<Item> items){
    //some code
    //some code
    //some code
    var sumPrice = items.sum(p => p.Price);
    var canApplyDiscount = CanApplyDiscount(sumPrice);
    if(canApplyDiscount == true)
        return ApplyDiscount(sumPrice);
    else
        return sumPrice;
}
public bool CanApplyDiscount(decimal price){
    if(price >= 100)
        return true;
    else
        return false;
}
public decimal ApplyDiscount(decimal price)
{
    return price - price/10;
}
```

همان طور که مشاهده می‌کنید در هر دو نمونه قانون تخفیف 10 درصدی اعمال شده و به درستی کار می‌کند، اما نمونه Explicit این قانون را بسیار شفاف تر به نمایش گذاشته است.

معایب Implicit بودن مفاهیم:

- سخت شدن یادگیری Domain و Model
- سخت شدن نگهداری کد
- برنامه نویسان جدیدی که به تیم ملحق می‌شوند ممکن است از وجود چنین مفاهیمی ناآگاه باشند و به دلیل شفاف نبودن این موارد تا مدت ها از وجود این قوانین ناآگاه بمانند
- پنهان بودن پیچیدگی های Domain در این موارد که در طولانی مدت باعث عدم توسعه پذیری پروژه می‌شوند

با تبدیل این موارد به مفاهیم Explicit می‌توان این معایب را در پروژه از بین برد.

آیا باید تمام موارد Implicit را تبدیل به Explicit کرد؟ خیر

همان طور که در دو نمونه بالا مشاهده کردید اگرچه مورد Explicit خواناتر بود اما باعث شد که کد بیشتری نوشته شود. پس اگر بخواهیم تمام موارد Implicit را تبدیل به Explicit کنیم بیش از حد به جزییات پرداخته ایم و تمام موارد غیر مهم را نیز به پروژه اضافه کرده ایم. بنابراین هر آن چیزی که مربوط به Core Domain است یا جزو قوانین کسب و کار محسوب می‌شود باید به صورت Explicit در کد و Model پیاده سازی شود.

خب برای شروع تا حدودی با DDD آشنا شدید. از اینجا به بعد جزییات DDD را به صورت تخصصی بررسی می‌کنیم.

فرآیند طراحی استراتژیک

هر کسب و کاری برای موفقیت و ادامه حیات ناگزیر باید یک استراتژی منحصر به فرد برای خود داشته باشد. معمولا این استراتژی حول محور مزیت رقابتی کسب و کار شکل می‌گیرد و تمام قوانین و فرآیند های کسب و کار بر اساس همین استراتژی تعریف می‌شوند. اما موضوع به همینجا ختم نمی‌شود، استراتژی سازمان، تعداد پرسنل سازمان، تعداد واحدهای سازمان، محل استقرار واحد ها در سازمان، بودجه سالیانه، شرکای تجاری و هر آن چیزی را که در شکل گیری و پیشرفت مسیر کسب و کار نقش دارد را مشخص می‌کند.
استراتژی همه این موارد را با تمرکز بر مزیت های رقابتی کسب و کار به نحوی مشخص می‌کند که در نهایت منجر به موفقیت کسب و کار شود.

تا قبل از به وجود آمدن تکنولوژی تمام کسب و کار ها استراتژی خود را با روش های سنتی اجرا می‌کردند. اما با به وجود آمدن نرم افزار ها و فراگیر شدن آنها روش های سنتی کنار گذاشته شدند و نرم افزار جایگزین شد. نکته ای که نباید فراموش کرد اینست که روش های سنتی از بین رفتند اما همچنان استراتژی و مزیت رقابتی کسب و کار سر جای خود قرار دارند. بنابراین نرم افزار ابزاری است برای اجرای استراتژی سازمان و قدرت نمایی مزیت رقابتی کسب و کار. بنابراین توسعه نرم افزار و طراحی معماری نرم افزار باید تحت تاثیر و تابع استراتژی سازمان باشد و بیشترین تلاش و هزینه در آن قسمتی از نرم افزار خرج شود که دقیقا مزیت رقابتی کسب و کار را شامل می‌شود.

از طرفی طراحی سیستم نرم افزاری در کسب و کارهایی که دامنه بزرگ و پیچیده ای دارند، کار دشواری است. در این سیستم ها معمولا، با پیشرفت پروژه و اضافه شدن نیازمندی های جدید کسب و کار، به پیچیدگی طراحی اضافه می‌کند. غافل شدن از این پیچیدگی ها می‌تواند باعث شکست خوردن طراحی و پیاده سازی سیستم های نرم افزاری شود. معمولا دامنه کسب و کار های بزرگ به قدری پیچیده است که نمی‌توان به کل دامنه به چشم یک واحد نگاه کرد، پس، نیاز است که دامنه کسب و کار در دو بخش مفهوم و پیاده سازی به قسمت های کوچک تر تبدیل شود.

بنابراین به عنوان معمار سیستم های نرم افزاری نیاز به تکنیک ها و روش هایی داریم که به ما کمک کنند بتوانیم دامنه کسب و کار های بزرگ و پیچیده را به نحوی مدل سازی کنیم که بتوانیم در آینده یک سیستم نرم افزاری قابل توسعه و قابل نگهداری داشته باشیم تا در خدمت کسب و کار باشد و اهداف کسب و کار را به سرانجام برساند.

طراحی استراژیک در DDD شامل تکنیک های است که به معمار نرم افزار کمک می‌کنند تا بتواند دامنه کسب و کار را به دامنه های کوچک تر تبدیل کند و تمرکز طراحی را بر روی قسمت هایی بگذارد که مهم ترن قسمت های کسب و کار و استراتژی آن است و اهداف کسب و کار را شامل می‌شوند و راه کسب درآمد کسب و کار هستند.

انتخاب استراتژی های درست، به معمار نرم افزار کمک می‌کند مدل سازی بهینه ای از دامنه کسب و کار انجام دهد تا کسب و کار مورد نظر بتواند از طریق سیستم نرم افزاری به اهداف خود برسد.

در ادامه مراحل طراحی استراتژیک را در طراحی نرم افزار معرفی می‌کنیم.

اولین مرحله در طراحی استراتژیک شناسایی Subdomain ها است که به کمک آن می‌توانید فضای کسب و کار را به قسمت های کوچکتر تبدیل کنید که مهم ترین قسمت، Core Subdomain خواهد بود که بیشترین تلاش و هزینه برای این قسمت هزینه می‌شود، سپس یک سری قسمت ها Support Subdomain ها خواهند بود که

برای عملکرد درست Core Subdomain وجودشان ضروری است ولی می‌توان در صورت وجود یک سری محدودیت های سازمان (مثل کمبود بودجه)، توسعه آنها را برونسپاری کرد. در نهایت باقی قسمت ها Generic Subdomain ها خواهند بود که قسمتی از کسب و کار را شامل می‌شوند که معمولا در بسیاری از کسب و کار ها وجود دارند و به سطحی از بلوغ رسیده اند که استاندارد های مشخصی برای آن ها وجود دارد و هزینه کردن برای توسعه آنها مقرون به صرفه نیست پس می‌توان به جای توسعه، از محصولات موجود در بازار استفاده کرد.

به فرض مثال در یک فروشگاه اینترنتی سیستم مدیریت سفارشات، Core Domain است. سیستم پشتیبانی مشتریان یک Support Domain و سیستم پرداخت اینترنتی یک Generic Domain.

اینکه کدام قسمت از کسب و کار Core Subdomain است یا Support Subdomain و یا Generic Subdomain کاملا توسط استراتژی سازمان مشخص می‌شود، چرا که، توسط استراتژی سازمان مشخص می‌شود که مزیت رقابتی کسب و کار کدام قسمت است، توسط استراتژی سازمان مشخص می‌شود که کدام قسمت ها از اهمیت کمتری برخوردار هستند اما وجودشان برای حیات مزیت رقابتی ضروری است و توسط همین استراتژی مشخص می‌شود کدام قسمت ها نیازی به تمرکز و پیدا کردن راه حل ندارند چون قبلا راه حل های آن ها کشف شده و می‌توان از راه حل های آماده استفاده کرد. و در نهایت توسط استراتژی سازمان، بودجه و هدف های پیش رو مشخص می‌شود و معمار نرم افزار با همه این موارد می‌تواند تعداد و نوع Subdomain ها را مشخص کند.

دومین مرحله در طراحی استراتژیک شناسایی Bounded Context ها است. برای شناخت بهتر کسب و کار به کمک شناسایی Subdomain ها فضای کسب و کار را به قسمت های کوچک تر تبدیل می‌کنیم و میزان هزینه و تلاش برای قسمت های مختلف را می‌شناسیم و تا مقدار زیادی از مسیر پیش روی توسعه نرم افزار را مشخص می‌کنیم، اما برای شناخت بهتر از هر Subdomain باید داخل هر کدام را نیز به بخش های مشخص تقسیم کنیم. معمولا سیاست میزان هزینه کرد برای هر Bounded Context داخل یک Subdomain یکسان است ولی می‌تواند نسبت به باقی Bounded Context های موجود در باقی Subdomain ها با توجه به Core بودن یا Support بودن آنها متفاوت باشد. مزیت دیگری که مشخص کردن Bounded Context ها دارد اینست که چند تیم به صورت همزمان و موازی می‌توانند بر روی Bounded Context های متفاوت کار کنند و آن Subdomain زودتر تکمیل شود.

در واقع Bounded Context ها واحد های مستقل با مرزهای مشخص در هر Subdomain هستند که قسمتی از وظایف مربوط به آن Subdomain را به عهده دارند. طراحی استراتژیک در اینجا مشخص می‌کند که وظایف هر Subdomain به چند دسته باید تقسیم شوند و مدل سازی آن وظایف به چه شکلی باید باشد و اینگونه هر دسته بندی به یک Bounded Context تبدیل می‌شود.

به فرض مثال در یک فروشگاه اینترنتی در Core Domain می‌توانیم سه Bounded Context برای ثبت سفارش، تحویل سفارش و وضعیت مالی سفارش داشته باشیم.

یک تصمیم استراتژی در مورد شناسایی Bounded Context ها در زمان شروع پروژه است. اگر در شروع نمی‌دانید باید چه تعداد Bounded Context داشته باشید، می‌توانید کل سیستم را به شکل یک Bounded Context واحد در نظر بگیرید و طراحی و توسعه را شروع کنید. سپس همچنان با پیشرفت پروژه دو اتفاق مهم رخ می‌دهد هم آن Bounded Context وسیع تر می‌شود و هم تیم توسعه بزرگتر می‌شود، که این دو اتفاق باعث کند شدن پیشرفت پروژه می‌شود، چرا که با بزرگ شدن Model و افزایش تعداد اعضای تیم دیگر

همگام بودن اعضای تیم با یکدیگر و با توسعه پروژه کار دشواری است. در اینجا می‌توانید Bounded Context را بر اساس دسته بندی عملکرد های سیستم و بر اساس Model های موجود در پروژه به چند Bounded Context دیگر تقسیم کنید و همچنین چندین تیم توسعه تشکیل بدهید و هر کدام مسئول توسعه یک Bounded Context باشد.

سومین مرحله در طراحی استراتژیک مشخص کردن زبان مشترک داخل هر Bounded Context است. از آنجایی که قبل تر هم اشاره کردیم زبان مشترک و ادبیات مشخص می‌تواند تاثیر بسزایی در موفقیت یا شکست پروژه نرم افزاری داشته باشد، پس از روی اهمیت آن یکی از مراحل در طراحی استراتژیک مشخص کردن زبان مشترک و ادبیات داخل هر Bounded Context است.

چهارمین مرحله در طراحی استراتژیک مشخص کردن نوع ارتباط و نحوه تعامل بین Bounded Context ها است. این مرحله از مهم ترین مراحل در طراحی استراتژیک است و طراح نرم افزار با مشخص کردن Context Mapping یک نقشه کلی از ارتباطات و تعاملات بین Bounded Context های داخل هر Subdomain ارائه می‌کند. این راه حل یکی از راه حل های استراتژیکی است که به تیم ها کمک می‌کند با پروتکل های مشخص با یکدیگر ارتباط برقرار کنند و اینگونه با صرفه جویی در زمان، کمک می‌کند تا پروژه ها سریع تر و با چالش کم تر توسعه پیدا کنند. همچنین باعث می‌شود تیم ها از Bounded Context های دیگر و فضای کسب و کار شناخت بهتری داشته باشند که این خود به پیشرفت پروژه نرم افزاری کمک می‌کند.

پنجمین مرحله در طراحی استراتژیک تقطیر سازی دامنه(Distillation) است. تقطیر کردن به معنی استخراج کردن مواد با ارزش موجود در یک ماده است. این اصطلاح بیشتر در علم شیمی شناخته شده است، اما Eric Evans از آن برای بخش مهمی از طراحی استراتژیک در DDD استفاده کرده است. در رویکرد DDD تقطیر دامنه به معنی استخراج قسمت های مهم Core Domain برای قابل فهم تر شدن آن می‌باشد. قسمت های مهم Core Domain دقیقا همان قسمت هایی هستند که مزیت رقابتی کسب و کار را شامل می‌شوند و یا ارکان اصلی کسب و کار هستند. به عنوان مثال می‌توان به قابلیت هایی که برای مشتریان فراهم می‌کند، مشکلاتی که از مشتریان برطرف می‌کند و یا قوانین کسب و کار اشاره کرد.

در واقع در یک پروژه نرم افزاری شناسایی Sub Domain ها، شناسایی Bounded Context ها و مرز بین آنها و Model سازی کردن Domain بخشی از این تقطیر سازی است.

برای درک بهتر تقطیر سازی در DDD می‌توان فرآیند مدل سازی را در نظر گرفت. شما برای توسعه هر قسمت از نرم افزار نیاز به تحلیل آن قسمت در کسب و کار واقعی را دارید و پس از شناخت نیازمندی های آن قسمت از کسب و کار به همراه قوانینی که دارد و نحوه رفتار آن قسمت و تمام موارد مهم مربوط به آن قسمت یک مدل سازی انجام می‌دهید و سپس آن Model را در نرم افزار توسعه می‌دهید. پس می‌توانیم بگوییم شما قسمتی از دانش مربوط به یک کسب و کار را تقطیر کرده اید و حاصل این تقطیرسازی همان Model به دست آمده است. حال با پیشرفت پروژه و عمیق شدن افراد نسبت به دامنه این تقطیر سازی هم عمیق تر می‌شود و نتایج با ارزش تری نسبت به مراحل قبل تر به دست می‌آید. این نتایج با ارزش تر همان Model هایی هستند که تغییر می‌کنند و Refactor می‌شوند.

چرا تقطیر سازی در DDD اهمیت دارد؟ در سیستم های بزرگ دامنه کسب و کار و قوانین آن به قدری گسترده هستند که باعث پیچیده شدن Core Domain می‌شوند. حال اگر Core Domain پروژه نرم افزار شما پیچیده باشد، فهمیدن آن برای اعضای تیم سخت می‌شود، اگر اعضای تیم نتوانند Core Domain را به خوبی درک کنند در اعمال تغییرات نیز ناتوان خواهند بود و در نهایت هیچ تخمینی از هزینه و تاثیرات تغییرات نخواهند

داشت. پس با تقطیر Core Domain قسمت های حیاتی آن را جدا می‌کنیم تا Core Domain قابل فهم تر برای ضینفعان پروژه داشته باشیم. نتیجه این کار شناخت راحت تر Core Domain توسط تمام اعضای تیم و به دنبال آن توسعه با سرعت بالاتر و دقت بیشتر است.

فرآیند مدل سازی

مدل سازی قلب تپنده رویکرد DDD است. مدل سازی به روش های مختلف و از زاویه دید های مختلف انجام می‌شود. فرآیند مدل سازی از تحلیل Domain آغاز می‌شود و هدف رویکرد DDD این است که Model تحلیل شده همسو با طراحی سیستم و کد باشد. در واقع طراحی یک انتزاع از Domain است و Model نمونه پیاده سازی شده این طراحی است پس اگر Model تحلیل شده با طراحی تفاوت داشته باشد بدین معنی است که انتزاع برگرفته شده از Domain به درستی پیاده سازی نشده است.

تعریف Subdomain:

هر نرم افزاری بر اساس یک یا چند نیاز به وجود می‌آید و راه حلی برای برطرف کردن آن نیازها ارائه می‌کند. به فضایی که این نیازها را دربر می‌گیرد، فضای مساله گفته می‌شود. فضای مساله گاهی اوقات آن قدر بزرگ است و تعداد مشکلات و نیازهای زیادی را در خود می‌گجاند که باعث می‌شود به هم ریخته و پیچیده به نظر برسد. قطعا اگر شما با یک فضای پیچیده و به هم ریخته روبرو باشید و بخواهید با همان پیچیدگی و به هم ریختگی توسعه پروژه را شروع کنید و ادامه دهید نمی‌توانید نرم افزاری ارائه کنید که راه حل های مناسبی برای آن مشکلات و نیازها باشد.

برای سر و سامان دادن این فضای پر از پیچیدگی و به هم ریختگی باید نیازمندی ها و مشکلات موجود را دسته بندی کنید. نیازهایی که با یکدیگر ارتباط معنایی دارند را در یک دسته قرار دهید و برای هر دسته یک اسم انتخاب کنید که نشان دهنده دلیل انتخاب این دسته است. حال با نگاه کردن به فضای مساله اثری از به هم ریختگی وجود ندارد و تا حدی پیچیدگی کاهش یافته است. به این روش که باعث شفاف تر شدن فضای مساله می‌شود Subdomain Modeling و به هر کدام از آن دسته ها Subdomain گفته می‌شود.

از دیدگاه DDD سه نوع Subdomain داریم:

- Core Subdomain
- Supporting Subdomin
- Generic Subdomain

Core Subdomain: در این دسته مهم ترین نیازها قرار می‌گیرند. همان طور که در قسمت ابتدایی توضیح دادیم این دسته مهم ترین دسته است که دلیل اصلی وجود نرم افزار است. راه حل هایی که برای برطرف کردن نیازها و مشکلات موجود در این دسته ارائه می‌شوند در واقع همان مزیت رقابتی است که قرار است توسط این نرم افزار برای صاحب کسب و کار نسبت به رقبا ایجاد کند. بیشترین تلاش و بیشترین هزینه باید به این دسته اختصاص داده شود. به عبارت دیگر می‌توان گفت این دسته محل درآمدزایی کسب و کار است. برای مثال در یک فروشگاه اینترنتی که نیازهای زیادی را در حوزه فروش یک کسب و کار برطرف می‌کند، سیستم مدیریت سفارشات جز همین دسته حساب می‌شود. چرا که باید تعداد زیادی سفارش را در کمترین زمان ممکن با دقت بالا و عملکرد بهینه پردازش کند و این خود یک مزیت رقابتی نسبت به باقی رقبا و نقطه ورود پول به کسب و کار است.

Supporting Subdomain: قاعدتا نیازمندی های موجود در دسته Core Subdomain یک سری نیازمندی هایی دارند که کمتر شامل مزیت رقابتی می‌شوند، پس این نیازمندی ها در دسته Supporting Subdomain قرار می‌گیرند. وجود این نوع دسته بندی برای پاسخ دادن به نیازمندی های Core Subdomain ضروری است.

برای مثال در همان فروشگاه اینترنتی سیستم انبارداری یک Supporting Subdomain است که نیاز نگهداری محصولات را برای سیستم مدیریت سفارشات انجام می‌دهد.

Generic Subdomain: یک سری نیازمندی ها در هر کسب و کاری وجود دارند که در بین تمام کسب و کارها مشترک هستند و یک جزو نیازهای روزمره هر کسب و کاری هستند. به عبارت دیگر ارتباطی با حوزه فعالیت کسب و کار ندارند. معمولا راه حل های موجود در این دسته دارای استاندارد های مشخصی هستند که در بین اکثر کسب و کار ها یکسان می‌باشند. در واقع می‌توان گفت نیازها و راه حل هایی که در این دسته قرار می‌گیرند ابزاری هستند در اختیار Core Domain و Supporting Subdomain برای آنکه بتوانند راه حل های خود را بهتر اجرا کنند.

برای مثال در همان فروشگاه اینترنتی سیستم ارسال ایمیل به مشتریان برای دلایل متفاوت یک نوع Generic Subdomain حساب می‌شود. چرا که زمان زیادی است که استاندارد مشخصی برای این موضوع معرفی شده و تمام کسب و کار ها از همین استاندارد استفاده می‌کنند.

سوال: از هر نوع Subdomain به چه تعداد می‌توانیم در پروژه داشته باشیم؟

معمولا در کسب و کارهای بزرگ دنیا بیش از یک منبع درآمدی وجود دارد و این منابع درآمدی ممکن است در فضای های جداگانه قرار دارند پس طبیعی است که در یک کسب و کار بزرگ چند Core Subdomain داشته باشیم، که هر کدام نیازهای هر کدام از منبع های درآمدی را شامل می‌شود.

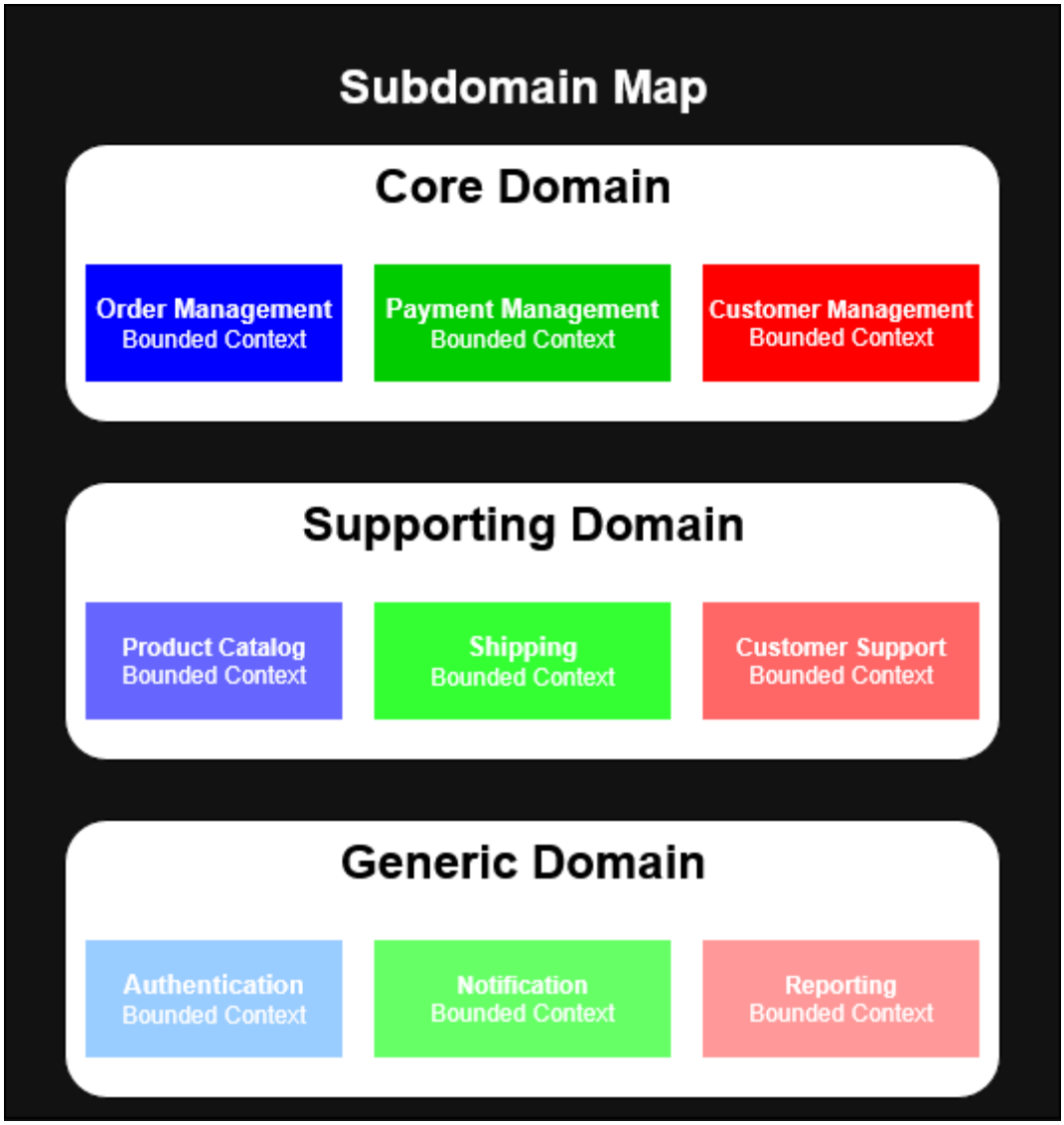
تفاوتی ندارد که یک Core Subdomain داشته باشیم یا چند تا، در هر صورت تعداد Supporting Subdomain ها بیشتر از یک خواهد بود. چرا که معمولا نیازهای Core Subdomain زیاد هستند و تفاوت معنایی آنها باعث به وجود آمدن دسته های مختلف Supporting Subdomain ها می‌شود.

همانند Supporting Subdomain ها میتوان چندین Generic Subdomain در هر پروژه ای داشت تا نیازهای مختلف دسته های دیگر را برطرف کنند.

توسعه یا برونسپاری و یا استفاده از محصولات آماده؟

از آنجایی که برای Generic Subdomain ها راه حل ها با استانداردهای مشخص در قالب یک سری محصولات نرم افزاری ارائه شده است و برای آنکه بتوان تمرکز و تلاش تیم های توسعه را معطوف به Core Subdomain کرد، می‌توان به جای توسعه Generic Subdomain ها از همین محصولات آماده استفاده کرد.

در صورت محدود بودن منابع اعم از: نیروی انسانی، مالی و یا زمانی می‌توان در صورت موجود بودن محصولات آماده مربوط به Supporting Subdomain ها به جای توسعه از محصولات آماده استفاده کرد یا توسعه آن را برونسپاری کرد. در غیر اینصورت بهتر است توسعه، توسط تیم توسعه خود کسب و کار انجام شود.



نمونه ای از انواع Domain ها در یک فروشگاه اینترنتی

ارتباط Bounded Context و Subdomain ها

هر Subdomain تعداد نیازهای زیادی را در خود جای می‌دهد که ممکن است توسعه راه حل ها و پیاده سازی نرم افزارهای مربوطه نیاز به چندین تیم توسعه داشته باشد. بنابراین داخل هر Subdomain مجدد نیازمندی های موجود را از لحاظ معنی دسته بندی می‌کنیم و یک مرزی بین هر دسته بندی ترسیم میکنیم. هر دسته بندی داخل Subdomin یک Bounded Context نامیده می‌شود. در بخش بعد کامل در مورد Bounded Context ها صحبت خواهیم کرد.

تعریف Bounded Context:

همانطور که در قسمت های قبل اشاره شد، هر نرم افزار حول محور فضای یک مساله شکل می‌گیرد سپس فضای آن مساله مدل سازی می‌شود و پروژه نرم افزاری در راستای آن Model توسعه پیدا می‌کند. Modelیی که در مرحله ظهور مساله شکل می‌گیرد بسیار کوچک و ساده است، از طرفی این Model باید در تمام طول عمر نرم افزار یکپارچه باشد. هر اصطلاح و مفهومی که در این پروژه نرم افزاری کشف می‌شود باید در تمام قسمت ها معنی یکسانی داشته باشند. با توجه به این موارد یک سوال مهم شکل به وجود می‌آید: **آیا این کار شدنی است؟ پاسخ: خیر نشدنی است.** اما چرا؟

بدیهی است که در مرحله ظهور مساله و مدل سازی اولیه فضای مساله، Model به دست آمده کوچک و ساده باشد اما با پیشرفت پروژه و پرورش فضای مساله و به بلوغ رسیدن Model به تدریج فضای مساله نیز رشد می‌کند و روز به روز بزرگتر می‌شود و آن Modelیی که در مرحله اولیه کوچک و ساده بود پس از مدتی تبدیل به یک Model بزرگ و پیچیده می‌شود و به دنبال آن تعریف هر مفهوم و عبارتی آن قدر بزرگ و جامع می‌شود که باعث ایجاد ابهام و پیچیدگی در مفاهیم می‌شود. فرض کنید در Model شما عبارتی به نام X وجود دارد، X ممکن است از نظر هر کدام از تحلیل گران سیستم معنی متفاوتی داشته باشد و به دنبال آن هر کدام از تعریف ها یک سری قانون اعتبار سنجی اضافه می‌کند که ممکن است بعضی از این قوانین با یکدیگر در تضاد باشند، این یعنی پیچیدگی همراه با ابهام. حفظ این پیچیدگی و کنار آمدن با آن در پروژه دو نتیجه ممکن خواهد داشت: یا پروژه نرم افزاری در نهایت با شکست مواجه می‌شود و یا پروژه با هزینه های سرسام آور به توسعه خود ادامه خواهد داد. اما راه حل مطمئنی که هم احتمال موفقیت پروژه را بیشتر کند و هم از لحاظ هزینه مقرون به صرفه تر باشد چیست؟

خب روشی که DDD برای این مشکل در نظر گرفته، طراحی Bounded Context ها است. در واقع در این روش طراح نرم افزار هر Domain را به قسمت های مختلف تقسیم می‌کند هر قسمت با قسمت دیگر مرزی مشخص و زبان مخصوص خود را دارد و مستقل از قسمت های دیگر رشد و توسعه پیدا می‌کند. به هر کدام از این قسمت ها یک Bounded Context گفته می‌شود. کار بسیار دشوار در طراحی Bounded Context ها مشخص کردن مرز بین هر Bounded Context با Bounded Context دیگر است. مشخص کردن Ubiquitous Language و وظایف و نقش هر Bounded Context در Domain مربوطه می‌تواند به طراح در مشخص کردن هر چه بهتر مرزها کمک کند.

با طراحی Bounded Context ها یک سیستم پیچیده به چند سیستم با پیچیدگی کمتر تقسیم بندی می‌شود که هر قسمت مستقل از قسمت های دیگر تکامل پیدا می‌کند و هر کدام بر روی جنبه های مختلف Domain کار می‌کند.

یک عبارت ممکن است در چند Bounded Context وجود داشته باشد و در هر Bounded Context تعریف متفاوت تری نسبت به دیگری داشته باشد. پس به یاد داشته باشید که Model هر Bounded Context باید داخل همان Bounded Context یکپارچه و مطابق با مرزهای آن باشد و نباید از مسائلی که بیرون آن Bounded Context وجود دارند تاثیر بپذیر. مثلا اگر عبارت X در Bounded Context معنی متفاوت تری نسبت به آن چه در Bounded Context فعلی تعریف شده، نباید در Bounded Context فعلی تاثیری داشته باشد.

تقسیم Domain به چند Bounded Context و مشخص کردن مرز بین آنها به معنی عدم ارتباط بین آنها نیست بلکه آنها باید برای پیشرفت در راستای هدف Domian با یکدیگر ارتباط و تعامل داشته باشند. نحوه ارتباط Bounded Context ها با یکدیگر پیچیدگی های خاصی دارد که DDD برای حل این مشکل و برای برقراری ارتباط و تعامل بین Bounded Context ها چند نوع روش برقراری ارتباط بین Bounded Context ها معرفی کرده است که شامل موارد زیر می‌باشد:

- رابطه Shared Kernel
- رابطه Customer/Supplier
- رابطه Conformist
- رابطه Anti-Corruption Layer
- رابطه Separated Ways
- رابطه Open Host Service

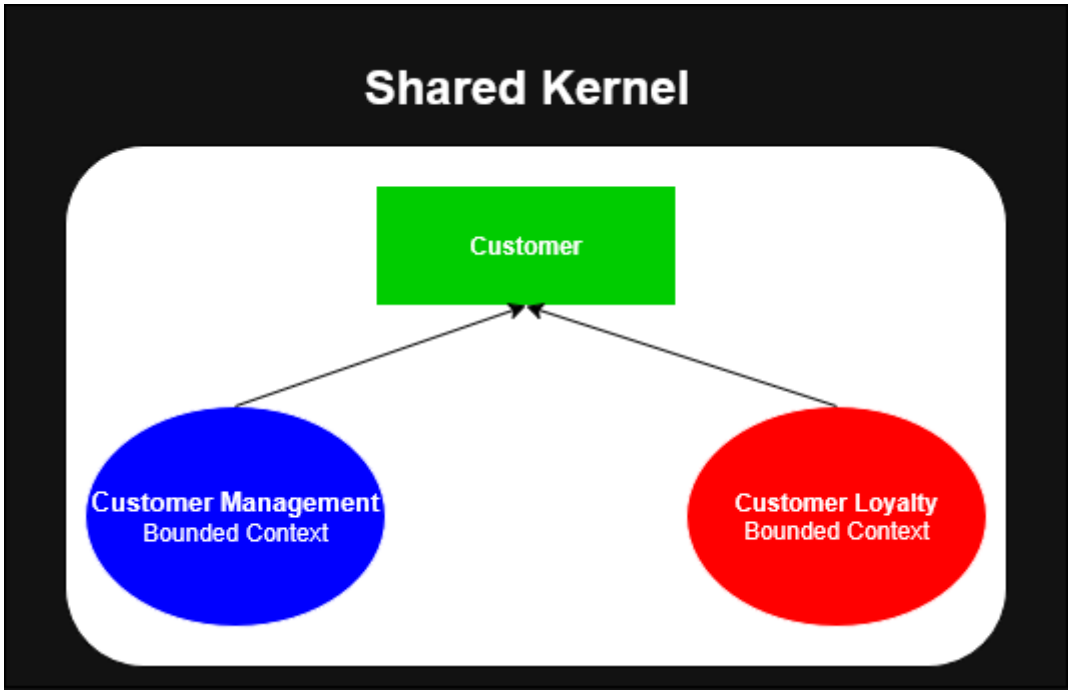
- رابطه Partnership

در ادامه هر کدام از این روش ها را تعریف و بررسی می‌کنیم.

تعریف Shared Kernel:

اگر چند Bounded Context داشته باشید و یک سری از مفاهیم و اشیا مشترک در این Bounded Context ها مورد استفاده باشند، برای پیاده سازی دو راه حل دارید یا باید تمام آن اشیا را در هر دو Bounded Context پیاده سازی کنید و یا این اشیا مشترک را در یک Bounded Context جدا ایجاد و نگهداری کنید و باقی Bounded Context ها برای استفاده از آن اشیا به Bounded Context جدید وابسته باشند و از آن استفاده کنند. به این Bounded Context جدید که اشیا مشترک Bounded Context های دیگر را نگهداری می‌کند Shared Kernel می‌گویند. و آن مفاهیم مشترک اجزایی از Domain هستند، مثل داده ها و یا سرویس هایی که در قسمت های مختلف سیستم از آن ها استفاده می‌شود.

برای مثال می‌توان به مفهوم مشتری در سیستم های نرم افزاری اشاره کرد. اکثر سیستم های نرم افزاری برای خدمت رسانی به یک سری مشتری پیاده سازی شده اند پس مفهوم مشتری در این نوع سیستم ها همیشه وجود دارد و اگر این سیستم نرم افزاری شامل چند Bounded Context باشد احتمالا تعدادی از آنها با موجودیت مشتری سروکار دارند، پس ما در این موقعیت می‌توانیم یک Shared Kernel بسازیم و موجودیت مشتری در آن تعریف کنیم و پس از آن هر Bounded Contextیی که نیاز به دسترسی به مشتری داشت می‌تواند از Shared Kernel استفاده کند.



Shared Kernel Example in an Online Shop

اما استفاده از Shared Kernel مزایا و معایبی دارد. شما با در نظر گرفتن این مزایا و معایب باید تصمیم گیری کنید آیا استفاده از این روش برای پروژه شما مناسب است یا خیر.

مزایا:

- صرفه جویی در کدنویسی و زمان: وقتی از Shared Kernel استفاده می‌کنید، دیگر مفاهیم مشترک را در چند جا کدنویسی نمی‌کنید و همین امر باعث کمتر شدن کدها و سریع تر پیش رفتن پروژه می‌شود.
- حفظ یکپارچگی Domain: اگر آن مفاهیم مشترک اجزای Domain باشند و نیاز باشد که حتما در تمام قسمت های مختلف سیستم یک تعریف با ویژگی های مشخص و ثابت داشته باشند، الگوی Shared Kernel کمک می‌کند تا یکپارچگی این مفاهیم در کل سیستم حفظ شود.
- کمک به درک بهتر Domain و کد: از آن جایی که برای این اشیا مشترک فقط یک تعریف در Shared Kernel داریم پس یک شی چندین بار در Bounded Context های مختلف پیاده سازی نمی‌شود، و همین موضوع که یک تعریف فقط در یک قسمت از پروژه وجود دارد باعث درک بهتر آن شی در کد و Domain می‌شود.

معایب:

- ایجاد وابستگی زیاد در پروژه: اگر قرار باشد تعدادی Bounded Context به یک Shared Kernel وابسته باشند، پس هر تغییری در Shared Kernel به معنی تغییر در تمام Bounded Context های وابسته است.
- نگهداری و توسعه با هزینه بالا: از آن جایی که مفاهیم Domain مداوم در حال تکامل هستند پس مفاهیمی هم که در Shared Kernel تعریف می‌شوند دچار تغییر و تحول می‌شوند. پس هر تیمی که بخواهد این تغییرات را در Shared Kernel اعمال کند ابتدا باید تمام Bounded Context ها را در جریان این تغییر بزارد و با تیم های توسعه مختلفی هماهنگی انجام شود که این کار هم زمانبر است و هم فرآیند توسعه را کند می‌کند.

توصیه ای برای استفاده از Shared Kernel: سعی کنید تا جای ممکن Shared Kernel را کوچک نگه دارید و فقط مفاهیمی از Domain را در آن تعریف کنید که نیاز است یکپارچگی آنها در تمام قسمت های مختلف سیستم حفظ شود. همچنین برای Shared Kernel تست های مناسبی در Bounded Context های وابسته بنویسید تا بعد از تغییر Shared Kernel با اجرای تست ها از عملکرد صحیح Bonded Context های وابسته اطمینان حاصل کنید.

تعریف Customer/Supplier:

بعضی وقت ها دو Bounded Context باید یک فرآیند را با همکاری یکدیگر انجام دهند که طی این همکاری نیاز است یکی از Bounded Context ها دیتای مورد نیاز دیگری را تهیه کند. در واقع Bounded Context یی که برای انجام وظیفه خود وابسته به Bounded Context دیگری است. در طراحی معماری نرم افزار سیستمی که سرویس دهنده است در اصطلاح Upstream تلقی می‌شود و سیستمی که سرویس گیرنده است Downstream تلقی می‌شود. حال اگر در این نوع رابطه توافق شود که Upstream طبق نیاز Downstream سرویس بدهد یعنی Downstream نیازمندی های خود را مشخص کند و Upstream طبق آن نیازمندی ها سرویس های خود را توسعه دهد به این نوع رابطه بین این دو Bounded Context رابطه Customer/Supplier گفته می‌شود. در واقع Customer همان سیستم وابسته است و Supplier همان سیستمی که سرویس مورد نیاز Customer را

تهیه می‌کند. حالا چرا نام گذاری Customer/Supplier برای این نوع رابطه انتخاب شده است؟ چون بسیار شبیه به رابطه یک تامین کننده و یک مشتری است. فرض کنید شما یک فروشگاه مواد غذایی دارید و بیشتر مشتریان شما تقاضای یک برند خاصی از یک شکلات را دارند پس قاعدتا شما طبق نیاز مشتریان خود آن برند از شکلات را بیشتر در فروشگاه خود موجود می‌کنید. در واقع شما به عنوان یک تامین کننده مواد غذایی محصولات فروشگاه خود را طبق نیاز مشتریان خود تامین می‌کنید.

در رابطه Customer/Supplier در سیستمی که Supplier است نیازمندی های Customer اولویت بالاتری نسبت به تصمیمات Supplier دارد، به عبارت دیگر Supplier سعی می‌کند خود را با خواسته Customer وفق دهد. اما این نکته هم مهم است که نیازمندی های Customer نباید برای ساختار و قوانین Supplier مخرب باشند. درست است که در تصمیم گیری های سمت Supplier نیازمندی های Customer اولویت دارند اما هر چیزی که از طرف Customer درخواست شود ممکن است از سمت Supplier قابل ارائه نباشد پس باید با مذاکره بین دو تیم یک بالانسی ایجاد شود که هم نیازمندی های Customer تا جای ممکن پاسخ داده شوند و هم Supplier مجبور به نقض قوانین و محدودیت هایی که دارد نشود.

این نوع رابطه نیاز به هماهنگی و همکاری نزدیک دو تیم Customer و Supplier دارد تا با یکدیگر بر سر نوع سرویسی که قرار است تیم Supplier توسعه دهد و دیتایی که باید در این سرویس رد و بدل شود توافق کنند. به همین منظور توصیه می‌شود که برای ارتباط بهتر و موثرتر حتما از اعضای تیم Customer در جلسات Planning تیم Supplier حضور داشته باشند و هر زمان که اعضای تیم Supplier نیاز داشتند بتوانند با اعضای تیم Customer ارتباط داشته باشند.

تیم Supplier با تیم Customer قراردادی مشخص می‌کند تا تعداد سرویس های مورد نیاز و دیتای مورد نیاز در هر سرویس را مشخص کنند. از این پس تیم Supplier توسعه را شروع می‌کند و هر کجا نیاز به همفکری تیم Cuswtomer داشت این تیم باید همکاری کند. پس از آماده شدن سرویس ها و تحویل آنها به تیم Customer تیم Supplier باید به گونه ای از این قرارداد نگهداری کند که تغییری در آن ایجاد نشود چرا که تیم Customer برای انجام وظایف خود وابسته به این سرویس ها است و اگر مورد در این سرویس ها تغییر کند تیم Customer دچار مشکل می‌شود.

برای حفظ جامعیت و یکپارچگی قراردادی که بین دو تیم در نظر گرفته شده، توصیه می‌شود که در سمت تیم Supplier یک سری Acceptance Test روی این قرارداد نوشته شود. با نوشتن این تست ها از این پس تیم Supplier می‌تواند توسعه سیستم خود را بدون نگرانی ادامه دهد چرا که اگر قسمتی از پروژه دچار تغییر شود که این قرارداد را نقض کند از طریق پاس نشدن Acceptance Test ها می‌توان متوجه آن شد و قبل Deploy شدن پروژه و تحت تاثیر قرار گرفتن Customer جلوی آن را گرفت. حال اگر این تست ها به صورت تست اتوماتیک پیاده شوند که بسیار موثرتر هم خواهند بود.

<u>مزایا:</u>

- **وظایف مشخص:** تیم Customer می‌داند که برای انجام وظایف خود دقیقا به چه دیتایی نیاز دارد و باید آن دیتا را از طریق چه سرویس و از کدام تیم دریافت کند، از طرفی دیگر تیم Supplier می‌داند که سرویس که باید ارائه کند مربوط به کدام تیم است و برای کدام فرایند این سرویس استفاده خواهد شد.
- **ماژولار بودن:** هر بخشی از فرایند در چند سرویس پیاده سازی می‌شود و در صورت نیاز می‌توان آن سرویس را جایگزین کرد.

- **کپسوله سازی:** تیم Customer دیتای مورد نیاز خود را از طریق سرویسی که تیم Supplier ارائه داده به دست می‌آورد و کار خود را پیش می‌برد بدون آنکه از جزییات داخل سرویس باخبر باشد.

معایب:

وابستگی شدید بین دو تیم: از آنجایی که تیم Customer وابسته به تیم Supplier است اگر تیم Supplier سرویسی که قبلا به تیم Customer داده است را تغییر دهد فرایندهای تیم Customer مختل می‌شوند.

نیازمند هماهنگی بالا بین دو تیم: برای تحویل یک سرویس کارآمدی که تمام نیازهای Customer را پاسخ دهد باید هماهنگی های زیادی بین دو تیم انجام شود، همچنین بعد از تحویل سرویس ممکن است هماهنگی هایی برای حفظ جامعیت سرویس یا تغییر سرویس صورت پذیرد.

توصیه ای برای استفاده از Customer / Supplier: این نوع رابطه بین Bounded Context ها جایی خوب عمل می‌کند که هر دو Bounded Context توسط دو تیم توسعه ای در حال توسعه هستند که آن دو تیم تحت نظارت یک مدیر کار می‌کنند، چرا که در این صورت هدف های تعریف شده برای هر دو تیم در یک راستا هستند و راحت تر می‌توانند بایکدیگر هماهنگ باشند و در راستای نیازمندی های یکدیگر حرکت کنند.

تعریف Conformist:

زمانی که یک رابطه Upstream/Downstream بین دو Bounded Context وجود داشته باشد و تیم توسعه آن دو Bounded Context مدیران جدا داشته باشند یا در بخش های کاملا متفاوت یک سازمان بزرگ باشند یا اصلا در سازمان های جدا باشند احتمالا رابطه این دو Bounded Context نمی‌تواند از نوع Customer/Supllier باشد چرا که این دو تیم به احتمال زیاد هدف های متفاوتی برای توسعه دارند و نمی‌توانند سطح بالایی از همکاری و یکپارچگی را با یکدیگر داشته باشند. از همین رو نیازمندی های Downstream برای Upstream اهمیت کمتری دارند، پس تیم Upstream تمایلی برای ارائه سرویس های مورد نیاز به Downstream ندارد همچنین تمایلی برای مذاکره و همکاری و یکپارچگی هم با Downstream ندارد، چرا که هدف کاملا متفاوت و اولویت های مهمتری از نیازمندی های Downstream دارد. در چنین شرایطی که Upstream نمی‌تواند پاسخگوی نیازمندی های Downstream باشد سه راه حل وجود دارد:

- از بین بردن وابستگی بین Upstream و Downstream یا به عبارتی حذف رابطه Upstream/Downstream
- دریافت سرویس از Upstream و اعمال تغییر یا پردازش بر روی خروجی سرویس های Upstream به نحوی که یکپارچه با مدل سازی Downstream باشد
- تبعیت کردن Downstream از مدل سازی Upstream به نحوی که مدل سازی Downstream همانند مدل سازی Upstream باشد

به رابطه ای که Downstream تابع Upstream است Conformist گفته می‌شود. در این نوع رابطه Downstream مدل سازی Upstream را انتخاب می‌کند و طبیعتا Modelیی که Downstream با آن کار می‌کند رفتارها و ویژگی هایی دارند که Upstream برای آنها تعریف کرده است. در نتیجه توانمندی های Downstream محدود به توانمندی های مدل Upstream است و وابستگی Downstream به Upstream را عمیق تر می‌کند، به همین دلایل باید سعی کنید کمتر سراغ این نوع رابطه برای Bounded Context های خود بروید.

مزایا:

کاهش پیچیدگی در Downstream: از آنجایی که Model استفاده شده در Downstream تابع Model استفاده شده در Upstream است پس مدل سازی در Downstream انجام نمی‌شود که در نتیجه باعث کمتر شدن تعداد Model ها و کاهش پیچیدگی می‌شود

افزایش یکپارچگی بین دو Bounded Context: از آنجایی که فقط یک Model وجود دارد و یک تیم مسئول آن است، بنابراین یکپارچگی نیز بیشتر حفظ می‌شود

معایب:

وابستگی شدید: این نوع رابطه باعث وابستگی زیاد Downstream به Upstream می‌شود

کاهش خودمختاری Downstream: از آنجایی که Upstream ممکن است تغییراتی در Model به وجود آورد و سیستم Downstream کاملا تحت تاثیر این تغییرات قرار می‌گیرد و از این تغییرات بی خبر است نمی‌تواند در این موقعیت رفتار پیش گیرانه برای جلوگیری از بروز خطا داشته باشد. همچنین از آنجایی که Modelیی که Downstream به آن وابسته است متعلق به Upstream است اگر Downstream نیاز داشته باشد نمی‌تواند Model را تکامل بخشد و آن را پرورش دهد و در این زمینه باید کاملا تابع Upstream باشد

شباهت بین رابطه Shared Kernel و Customer/Supplier و Conformist در این است که هر سه رابطه بین Bounded Context هایی انتخاب می‌شوند که بر روی قسمت مشترکی از Domain کار می‌کنند. تفاوت بین این سه نوع رابطه در نحوه توسعه پروژه و تصمیم گیری ها مشخص می‌شود، به فرض مثال رابطه Shared Kernel همکاری عمیق بین دو تیم را به وجود می‌آورد در حالی که در رابطه Conformist یکپارچگی تیم تبعیت کننده با تیمی است که تمایلی به همکاری با این تیم را ندارد.

تعریف Anti-Corruption Layer:

در توسعه پروژه های نرم افزاری زمان هایی وجود دارند که دو سیستم نرم افزاری نیاز دارند تعاملات گسترده با یکدیگر داشته باشند تا یک فرآیند را به اتمام برسانند و به یکی از دلایل زیر هر کدام Model و زبان مخصوص خود را دارند

- هر دو سیستم در Domain های متفاوتی هستند
- هر دو سیستم در یک Domain هستند اما هر کدام در Bounded Context های جدا هستند
- هر دو سیستم در یک Domain و در یک Bounded Context هستند اما مدل سازی متفاوتی از Domain دارند

از طرفی برای آنکه دو سیستم بتوانند با یکدیگر تعامل داشته باشند باید ارتباطی بین Model دو سیستم وجود داشته باشد یا این ارتباط را ایجاد کنیم. هر سیستم مدل سازی مخصوصی از Domain مربوط به خود را دارد که یک سری معانی و مفاهیمی از Domian را داخل Model خود به همراه دارد. وجود داشتن ارتباط بین Model دو سیستم به این معنی است که هر سیستم یک سری از مفاهیمی که در دل Model سیستم مقابل قرار گرفته است را بشناسد و بتواند از طریق آن مفاهیم، ارتباط خود را با سیستم مقابل برقرار کند. البته که برقراری این ارتباط کاری دشوار و زمانبر است. اگر فرض کنیم که می‌توانیم با سختی این کار کنار بیاییم و زمان مورد نیاز را هم اختصاص دهیم باز یک ریسک وجود دارد، ریسک نشت Model یک سیستم به سیستم دیگر. نشست Model را به این شکل در نظر بگیرید که برای برقرار کردن ارتباط بین Model دو سیستم، سعی کنید

مفاهیم موجود در Model یک سیستم را داخل Model سیستم دیگر بگنجانید، که در این صورت این Model را خراب کرده اید چرا که آن مفاهیم متعلق به آن Model نیست و این کار باعث می‌شود که این Model کاملا از هدف و دلیل وجودی خود دور شود، که در این صورت این Model بی ارزش به حساب می‌آید.

تفاوت در Model یکی از مشکلات در تعامل بین سیستمی است، مشکل دیگر جایی به وجود می‌آید که دو سیستم برای برقراری ارتباط از تکنولوژی ها یا پروتکل های متفاوت استفاده می‌کنند. فرض کنید در این حالت یک سیستم برای آنکه بتواند با سیستم دیگر ارتباط برقرار کند بخواهد تکنولوژی مورد استفاده در آن سیستم را در زیرساخت خود پیاده سازی کند یک تکنولوژی جدید که پیچیدگی های مخصوص خود را هم همراه دارد به زیرساخت خود اضافه کرده است. این کار جایی سخت تر می‌شود که تکنولوژی ارتباطی سیستم دیگر قدیمی و تاریخ گذشته است و قاعدتا هیچ تیمی تمایل ندارد برای برقراری ارتباط بین سیستم خود با سیستم دیگر یک تکنولوژی تاریخ گذشته را به پروژه خود اضافه کند.

پس میتوانیم نتیجه بگیریم برقراری و حفظ تعامل بین دو سیستم نرم افزاری می‌تواند کاری پر هزینه و دشوار باشد، اما از آنجایی تعامل بین سیستم ها برای رسیدن به اهدافی است که دلیل به وجود آمدن نرم افزار هستند، برای سازمان و تیم های توسعه ارزش قابل توجهی دارد، پس ترجیح می‌دهیم این تعامل را حفظ و نگهداری کنیم، اما با روش هایی که این کار را برای تیم های توسعه راحت تر و کم هزینه تر کرده است.

یکی از بهترین راه حل ها برای این مشکلات ایجاد یک لایه جدید بین دو سیستم است. این لایه ارتباطات بین دو سیستم را ترجمه می‌کند و در عین حال سیستم سرویس دهنده بدون تغییر باقی می‌ماند و سیستم سرویس گیرنده تحت تاثیر Model و مشکلات تکنولوژی سیستم سرویس دهنده قرار نمی‌گیرد. یک لایه جدید بین دو سیستم تعریف کنید، این لایه با Model هر دو سیستم آشنا هست و با ارائه یک سری سرویس نیازهای هر دو سیستم را به صورت دو طرفه برطرف می‌کند بدون آن که نیاز به تغییر در هر کدام از سیستم ها باشد، فقط آن سیستم ها برای برطرف کردن نیازمندی خود نیاز دارند یکی سری سرویس از این لایه فراخوانی کنند. فرض کنید سیستم A نیاز به تعامل با سیستم B دارد، سیستم B یک سری رابط کاربری برای سرویس دهی به دنیای بیرون دارد اما این رابط های کاربری برای استفاده در سیستم A یک سری پیچیدگی ها دارند و Model سیستم A نیز با سیستم B متفاوت است. یک لایه مترجم بین این دو سیستم تعریف می‌کنید که یک سری سرویس به سیستم A می‌دهد و درون خود دیتایی که سیستم A از طریق سرویس ها به این لایه داده است به دیتای مورد نیاز سیستم B تبدیل می‌شود و سپس از طریق رابط های کاربری که سیستم B ارائه داده است برای آن ارسال می‌شود سپس پاسخی که از سیستم B دریافت می‌کند به شکل دیگری تغییر می‌دهد و به سیستم A بر می‌گرداند. بدین ترتیب هم نیاز سیستم A برطرف شده است و هم پیچیدگی های سیستم A داخل لایه مترجم پیاده سازی شده اند و از نشت Model سیستم B به سیستم A جلوگیری شده است. به این لایه مترجم Anti-Corruption Layer یا به اختصار ACL گفته می‌شود. لایه ACL یک لایه محافظ است که دربرابر نشت Domain Model سیستم سرویس دهنده به سیستم سرویس گیرنده محافظت می‌کند.

وظیفه ACL چیست؟ لایه ACL وظیفه دارد تا Model هر دو سیستم سرویس دهنده و سرویس گیرنده را درک کند و بشناسد و بتواند ترجمه بین این دو Model را به صورت دو طرفه انجام دهد و در نهایت یک Interface در اختیار سیستم سرویس گیرنده قرار دهد تا سیستم سرویس گیرنده بتواند به سرویس های مورد نیاز سیستم سرویس دهنده با زبان خود و بدون پیچیدگی ارتباط برقرار کند.

داخل لایه ACL یک Interface تعریف کنید، این Interface شامل تمام سرویس های مورد نیاز سیستم سرویس گیرنده می‌باشد که داخل هر سرویس پیچیدگی های سرویس های سیستم سرویس دهنده کنترل می‌شوند و Model سیستم سرویس گیرنده تبدیل به Model سیستم سرویس دهنده می‌شود و بالعکس. در واقع تمام پیچیدگی ها و ترجمه Model ها داخل سرویس های این Interface انجام می‌شوند، ولی روش بهتری هم وجود دارد، اگر از سیستم مقابل بیشتر شناخت داشته باشید می‌توانید Entity های موجود در سیستم سرویس دهنده را داخل ACL تعریف کنید و همراه آن، رفتارهای آن Entity هم پیاده سازی کنید، آنگاه داخل Interface خود تمام کارهای مدیریت پیچیدگی ها و ترجمه را داخل سرویس ها انجام نمی‌دهید بلکه بخشی از آن ها توسط این Entity ها انجام می‌شوند و این کار باعث می‌شود Interface شما پیاده سازی تمیزتری داشته باشد. با استفاده از این روش یک لایه ACL غنی پیاده سازی کرده اید که یک نمونه از سیستم سرویس دهنده را داخل ACL خود Model سازی کرده اید و به همین علت از قوانین، منطق، رفتارها، ساختارهای داده ای و ... سیستم سرویس دهنده آگاهی بیشتری دارید که با این سطح از شناخت از سیستم سرویس دهنده شاید بتوانید برای Model سازی سیستم مصرف کننده ایده بگیرید.

پیاده سازی لایه ACL می‌تواند ترکیبی از الگوهای Facade و Adapter و مترجم ها به همراه مکانیسم های ارتباطی (برای برقراری ارتباط بین سیستم ها) باشد.

الگوی Facade چیست؟ الگوی Facade در ارتباط با سیستم هایی استفاده می‌شود که استفاده از سرویس های آن سیستم پیچیده است، در واقع الگوی Facade یک پوششی است بر روی سرویس های پیچیده یک سیستم که تمام آن پیچیدگی ها توسط این الگو مدیریت و کنترل می‌شوند و به دنیای بیرون یک Interface ارائه می‌کند که هر استفاده کننده بدون نیاز از اطلاع داشتن از پیچیدگی ها فقط سرویس های آن Interface را فراخوانی می‌کند.

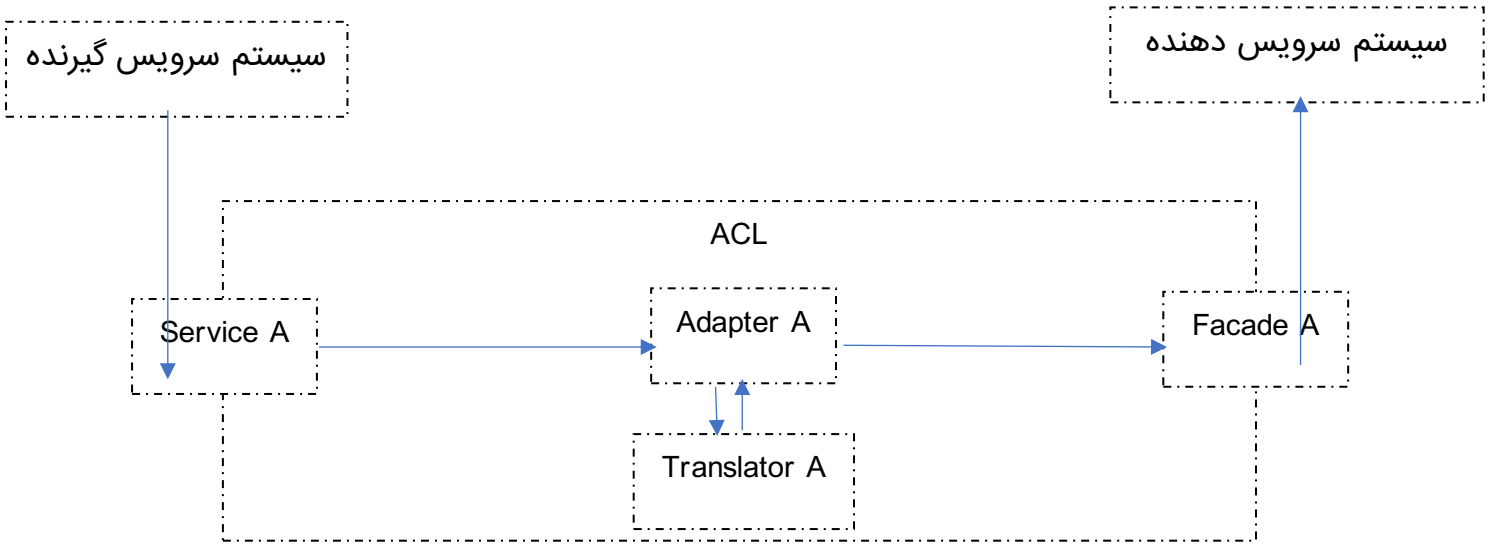
الگوی Adapter چیست؟ هر زمان دو Interface که بخواهند با یکدیگر ارتباط برقرار کنند اما زبان یکدیگر را متوجه نشوند، الگوی Adapter کار ترجمه بین این دو Interface را انجام می‌دهد.

در مبحث ارتباط بین سیستم ها معمولا نیاز داریم با سیستمی ارتباط برقرار کنیم که دارای Interface های پیچیده هستند و معمولا استفاده از این Interface ها هم راحت نیست. تبدیل Model یک سیستم به Model سیستم دیگر و استفاده همزمان از Interface های این سیستم کار بسیار دشواری است که به همین دلیل بهتر است از الگوی Facade به همراه Adapter برای این کار استفاده کنیم.

یک کلاس Facade داخل لایه ACL در واقع یک جایگزین از سیستم سرویس دهنده داخل سیستم سرویس گیرنده است که دسترسی به سیستم سرویس دهنده را برای ما آسان تر می‌کند. پس از آنجایی که جایگزین سیستم دیگر است باید مطابق با Model همان سیستم طراحی و پیاده سازی شود. با همه این تفاسیر می‌توان گفت که Facade اصلا عضوی از Bounded Context سیستم سرویس دهنده است که فقط برای استفاده راحت ما از سیستم سرویس دهنده داخل ACL سیستم سرویس گیرنده قرار گرفته است.

در عین حال که کلاس Facade مشکل پیچیدگی استفاده از سرویس های سیستم سرویس دهنده را برای ما برطرف می‌کند اما وظیفه ساخت و تهیه درخواست مناسب برای ارسال به سرویس های سیستم سرویس دهنده را ندارد، در اینجا الگوی Adapter این مشکل را حل می‌کند. یک کلاس Adapter وظیفه دارد تا درخواست سیستم مصرف کننده را تبدیل به درخواستی کند که سرویس های سیستم سرویس دهنده یا کلاس Facade آن را متوجه شوند.

برای درک بهتر می‌توان دیاگرام زیر را از ارتباطی که بین دو سیستم توسط ACL برقرار می‌شود ترسیم کرد:



وظیفه اصلی Adapter تبدیل درخواست سیستم سرویس گیرنده به درخواست قابل فهم برای سیستم سرویس دهنده است و ممکن است برای ساخت این درخواست نیاز به تبدیل یک سری مفاهیم از سیستم در حال توسعه برای سیستم سرویس دهنده داشته باشد که برای این کار می‌توانیم یک تا چند کلاس مترجم داشته باشیم تا این کار را برای Adapter انجام دهند.

دیاگرامی که در بالا ترسیم شده است کامل ترین نوع پیاده سازی شده از ACL است اما حالت ساده تری هم وجود دارد. اگر سیستم سرویس دهنده از لحاظ Domain Model شبیه به سیستم سرویس گیرنده باشد و مفاهیم یکسان یا شبیه به یکدیگر داشته باشند آنگاه احتمالا نیازی به Adapter و Translator نخواهیم داشت و فقط صرفا با استفاده از Facade پیچیدگی های سیستم سرویس دهنده را پوشش می‌دهیم.

چه زمانی از ACL استفاده کنیم؟ زمانی که دو یا چند Bounded Context دارید و هر کدام از آنها Model و زبان مخصوص خود را دارند و نیاز دارند با یکدیگر ارتباط برقرار کنند بدون آنکه Model هر کدام رو دیگری تاثییر بگذارد.

مزایا:

- **جلوگیری از نشت Model:** با ایجاد لایه ACL دیگر Model هیچ کدام از سیستم ها بر روی یکدیگر تاثییر نمی‌گذارند و از وارد شدن مفاهیم هر Model به Model سیستم دیگر جلوگیری می‌شود
- **کاهش پیچیدگی در سیستم در حال توسعه:** تمام پیچیدگی های مربوط به برقراری ارتباط بین دو سیستم و ترجمه Model ها داخل ACL انجام می‌شود، بنابراین از ورود این پیچیدگی ها به سیستم فعلی جلوگیری می‌شود

معایب:

- **ایجاد تاخییر در عملکرد سیستم:** با استفاده از ACL از آنجایی که یک لایه بین دو سیستم قرار می‌گیرد و این لایه ارتباطات بین دو سیستم را ترجمه می‌کند باعث ایجاد تاخییر در عملکرد سیستم می‌شود
- **افزایش حجم کار:** ایجاد لایه ACL باعث افزایش حجم کد نویسی و افزایش تعداد سرویس ها می‌شود و علاوه بر آن یک وظیفه جدید برای نگهداری و توسعه خود ACL به تیم تحمیل می‌کند

تعریف Separate Ways:

همانطور که در قسمت های قبل هم گفتیم حفظ تعامل و یکپارچه سازی بین سیستم ها کاری پرهزینه است، اما از آنجایی که هر سیستم نرم افزاری یک دلیل برای به وجود آمدن و توسعه پیدا کردن دارد، برای تحقق این هدف نیاز دارد تا تمام زیرسیستم های آن در راستای این هدف توسعه پیدا کنند، در راستای این هدف با یکدیگر یکپارچه شوند و در راستای این هدف با یکدیگر تعامل داشته باشند. بنابراین این هزینه را پرداخت می‌کنیم تا پروژه به هدف خود برسد.

هر چقدر Domain انتخاب شده برای توسعه نرم افزار بزرگتر باشد احتمالا تعداد زیر سیستم ها نیز بیشتر می‌شود و Bounded Copntext های بیشتری شناسایی می‌شوند. هر کدام از Bounded Context ها هدف خاص خود، زبان و مدل سازی مخصوص خود را دارد. در نهایت تمام این Bounded Context ها در راستای هدف اصلی سیستم در کنار هم توسعه پیدا می‌کنند. اما وقتی تعداد Bounded Context های یک پروژه نرم افزاری بالا می‌رود احتمال دارد تعدادی از Bounded Context ها در هیچ کدام از قسمت های Domain با یکدیگر اشتراکی نداشته باشند. به عبارت دیگر یک Domain شامل قسمت های مختلفی از یک کسب و کار واقعی است که بعضی از این قسمت ها از یکدیگر جدا هستند و هیچ ارتباطی با یکدیگر ندارند، اما عضوی از کسب و کار به شمار می‌آیند و برای ادامه حیات کسب و کار ضروری هستند.

در نتیجه تعامل و یکپارچگی عامل مهمی در موفقیت کسب و کار ها به حساب می‌آیند، اما از طرفی همیشه برقراری تعامل و یکپارچگی بین بعضی سیستم ها کمکی به موفقیت کل سیستم نرم افزاری نمی‌کند. پس برای این سیستم ها توصیه می‌شود هر کدام راه خود را در پیش گیرد و از زاویه دید خود به Domian نگاه کنند. به این نوع تصمیم طراحی که مشخص می‌کند دو یا چند Bounded Context یی که دنیای جداگانه ای دارند، هیچ ارتباطی با یکدیگر نداشته باشند و هر کدام راه خود را در پیش گیرند، Separate Ways گفته می‌شود.

مزایا:

- **کاهش هزینه ها**: وقتی تعامل و یکپارچگی بین چند Bounded Context را حذف می‌کنیم به دنبال آن وابستگی ها از بین می‌رود و در نتیجه دیگر بابت ارتباط برقرار کردن سیستم ها و هماهنگ بودن آن ها درباره بخش هایی از Domain هزینه ای پرداخت نمی‌کنیم.
- **افزایش سرعت توسعه**: از آنجایی که هر Bounded Context بدون توجه به بقیه Bounded Context ها مسیر خود را در پیش می‌گیرد فرآیند مدل سازی و توسعه با سرعت بیشتری پیش می‌رود.

معایب:

- **ریسک اتفاقات آینده**: از آنجایی که هر Bounded Context صرفا بر اساس نیاز خود فرآیند مدل سازی را پیش می‌برد و از مدل سازی بقیه Bounded Context ها آگاهی ندارد پس سطح طراحی انتزاعی در این Bounded Context کاهش پیدا می‌کند. چنانچه در آینده نیاز شود که این Bounded Context با بقیه Bounded Context ها ارتباط برقرار کند یا یکپارچه شود، این برقراری ارتباط و یکپارچگی پیچیده خواهد بود و احتمالا نیاز به Translator های پیچیده و بزرگ خواهد بود.

تعریف Open Host Service:

اگر در پروژه خود Bounded Contextیی دارید که یک سری سرویس به دو یا چند Bounded Context دیگر ارائه می‌کند و از طرفی برای هر Bounded Context سرویس جداگانه ای نوشته اید ولی کاری که آن سرویس ها انجام می‌دهند یکسان است، این Bounded Context سرویس دهنده کاندید مناسبی برای استفاده از روش Open Host Service است. در این روش توصیه می‌شود یک Interface تعریف کنید و داخل این Interface تمام سرویس هایی که Bounded Context های دیگر به آنها نیاز دارند را داخل این Interface قرار دهید. این Interface و سرویس هایی که داخل آن قرار می‌گیرند باید به صورت استاندارد تعریف شوند تا تمامی سرویس گیرنده ها بتوانند بدون مشکل از سرویس های مورد نیاز خود استفاده کنند.

معمولا اگر در یک سیستم برای توسعه سرویس های مورد نیاز سیستم های دیگر یک سری کد یکسان با منطق یکسان را داخل یک سری سرویس ها به صورت Copy/Paste می‌نویسید، باید یک بازنگری بکنید که اگر امکانش وجود دارد برای ارائه آن سرویس ها از روش Open Host Service استفاده کنید.

در دنیای واقعی می‌توان سرویس های پرداخت بانک ها را مثال زد که یک سری Api به تمام کلاینت ها برای امور پرداخت اینترنتی ارائه می‌کنند. مثال دیگر سرویس های احراز هویت یکپارچه در سیستم های بزرگ نرم افزاری است. همچنین در پیاده سازی Core Domain چنانچه بقیه Domain ها نیاز دارند یک سری دیتا از Core Domain دریافت کنند یا اگر Support Domain یا Generic Domainیی داریم که به چند Bounded Context سرویس های مشابه می‌دهند، می‌توانیم آن سرویس ها را از طریق روش Open Host Service ارائه کنیم.

توجه داشته باشید که سیستم های دیگر تحت تاثیر سیستم Open Host Service هستند پس اگر سرویس های ارائه شده دستخوش تغییر شوند تمام مصرف کنندگان تحت تاثیر قرار می‌گیرند، بنابراین برای تمام سرویس ها تست های مناسب بنویسید تا در صورت تغییر بخشی از سیستم مطمئن باشید منطق این سرویس ها همان طور که باید کار می‌کند. حتما از ابزار های Monitoring, Trace و Log مناسب استفاده کنید تا سرویس ها به طور دقیق ردیابی کنید تا قبل از به وجود آمدن مشکل از وقوع آن جلوگیری کنید یا در صورت به وجود آمدن مشکل آن را سریع تر شناسایی و برطرف کنید. و در نهایت مطمئن شوید که سیستم Host حتما Resilient باشد، یعنی اگر سیستم Host در اثر خطا از دسترس خارج شد حتما بتواند خودش را ریکاوری کند و به شرایط عادی بازگردد در غیر اینصورت تمام سیستم های سرویس گیرنده از سیستم Host دچار مشکل می‌شوند.

<u>مزایا</u>:

صرفه جویی در کدنویسی: از آنجایی که سرویس های که از طریق این روش ارائه می‌شوند برای استفاده چندین کلاینت توسعه داده شده اند پس دیگر برای هر کلاینت نیاز به نوشتن سرویس جدید نداریم.

کاهش پیچیدگی: با ارائه یک Interface به تمام مصرف کنندگان ضمن یکپارچه بودن با تمام سیستم ها و از آنجایی که یک سری سرویس یکسان برای تمام مصرف کنندگان داریم پس پیچیدگی سیستم نیز کاهش پیدا می‌کند.

<u>معایب</u>:

نگهداری سخت تر: معمولا استفاده کنندگان سرویس های ارائه شده در Open Host Service تعدادشان بیشتر از دو سیستم است و از طرفی تعداد دفعات فراخوانی این سرویس ها در لحظه مشخص نیست پس باید به

خوبی مانیتور بشوند تا در شرایط بحرانی بتوان قبل از اینکه سرویس ها از دسترس خارج بشوند تصمیم مناسب گرفته شود.

تعریف Partnership:

در بسیاری از پروژه های نرم افزاری شرایطی پیش می‌آید که یک فرآیند میبایستی توسط دو یا چند Bounded Context انجام شود. یعنی برای انجام این فرآیند Domain بیش از دو Bounded Context درگیر می‌شود، که در اینجا رابطه از نوع Customer/Supplier یا Conformist و ... برای ارتباط این Bounded Context ها مناسب نیست. در واقع در اینجا تمام Bounded Context ها باید با همکاری یکدیگر و با استفاده از یک زبان مشترک این فرآیند را به اتمام برسانند، که به این نوع رابطه، Partnership می‌گویند. در این روش تمام Bounded Context ها باید از Model یکدیگر شناخت داشته باشند و تعریف مشترکی از تمام مفاهیمی که در این فرآیند وجود دارند داشته باشند. در این روش فضای مساله (Domain) همان فرآیند است و تمام Bounded Context ها روی یک Model مشترک که از تحلیل فرآیند ساخته شده با یکدیگر همکاری می‌کنند.

در این روش Bounded Context ها بر روی یک یا چند پروتکل برای پیشبرد فرآیند با یکدیگر توافق می‌کنند. به طور مثال ممکن است دو Bounded Context در قسمتی از این فرآیند از طریق Api با یکدیگر ارتباط برقرار کنند و یک دیتایی رو رد و بدل کنند و در قسمتی دیگر از فرآیند همان دو Bounded Context از طریق ارسال Event دیتای مورد نیاز را برای یکدیگر ارسال کنند و فرآیند را پیش ببرند.

برای مثال در یک فروشگاه اینترنتی دو Bounded Context فروش و انبارداری می‌توانند رابطه از نوع Partnership داشته باشند. فرض کنید مشتری می‌خواهد سبد خرید خود را نهایی و پرداخت کند، ابتدا سیستم فروش یک Api از سیستم انبارداری فراخوانی می‌کنند تا مطمئن شود محصولات انتخابی مشتری در انبار موجود هستند سپس بعد از نهایی شدن خرید سیستم فروش یک Event برای هر محصول روی یک صف قرار می‌دهد تا سیستم انبارداری با دریافت آن Event متوجه شود که از هر محصول چه تعدادی به فروش رفته است.

برای آن که مطمئن باشید با استفاده از این روش فرآیند مورد نظر با موفقیت به پایان می‌رسد باید حتما تمام سیستم ها قابل اعتماد باشند. بنابراین تمام سیستم ها در این فرآیند باید بتوانند در صورت بروز مشکل و از کار افتادن به سرعت خود را ریکاوری کنند. همچنین هر سیستم باید بتواند در صورت از کار افتادن سیستم دیگر به کار خود ادامه دهد و از کار افتادن بقیه سیستم ها در عملکرد آن تاثیری نداشته باشد.

<u>مزایا:</u>

- **افزایش یکپارچگی**: تمام Bounded Context هایی که به صورت Partnership با یکدیگر همکاری می‌کنند بر روی یک Model مشترک کار می‌کنند و تعاریف مشترکی هم با هم دارند.
- **افزایش سرعت**: هر چند ممکن است در ابتدا برای رسیدن به یک Model مشترک و تعیین قراردادها و پروتکل های ارتباطی، سرعت تیم ها کاهش پیدا کند اما بعد از مشخص شدن این موارد، در طولانی مدت باعث افزایش سرعت تیم ها می‌شود.

معایب:

- **وابستگی زیاد:** در این روش، فرآیند زمانی با موفقیت به پایان می‌رسد که تمامی سیستم ها به درستی کار کرده باشند. اگر یک سیستم در حین انجام فرآیند به درستی کار نکند، ممکن است فرآیند ناتمام بماند یا با تاخیر زمانی به پایان برسد.
- **افزایش سربار:** از آنجایی که در این روش باید تمام سیستم ها قابل اعتماد باشند، نیاز است هر سیستم مکانیزمی برای ریکاوری کردن در برابر مشکلات درونی و مکانیزمی برای آسیب پذیر نبودن در برابر مشکلات بیرونی، پیاده سازی کنند. پیاده سازی این مکانیزم ها ممکن است یک سربار در اجرای فرآیند برای تمامی سیستم ها باشد.
- **تغییر پذیری سخت:** از آن جایی که یک Model بین تمام سیستم ها مشترک است، چنانچه یک سیستم نیاز به تغییر Model داشته باشد بدون هماهنگی و تایید بقیه تیم ها نمی‌تواند تغییری در Model اعمال کند.

تفاوت Shared Kernel با Partnership:

در Shared Kernel قسمتی از Model که در Domain هر دو Bounded Context حضور دارد به صورت مشترک بین دو Bounded Context تعریف می‌شود. هدف از این کار یکپارچه بودن قسمتی از Domain بین دو Bounded Context است تا ساختار دیتا، تعاریف و مفاهیم مربوط به آن قسمت از Domain در تمام Bounded Context های درگیر یکسان و ثابت باشد. اما همچنان Bounded Context های درگیر، در قسمت های دیگر Domain مستقل و خود مختار هستند. در واقع در این روش Bounded Context ها وابسته به Shared Kernel هستند و نه وابسته به یکدیگر. به عبارت دیگر اگر یک Bounded Conetxt به درستی کار نکند Bounded Context دیگر می‌تواند کار خود را به درستی انجام دهد.

در Partnership یک فرآیندی که باید توسط چند Bounded Context انجام شود به صورت یک Model مشترک ساخته می‌شود و تمامی Bounded Context ها از آن استفاده می‌کنند تا با یکدیگر برای انجام فرآیند تعامل داشته باشند. در این روش علاوه بر وابسته بودن Bounded Context ها به Model ، به یکدیگر نیز وابسته هستند. به عبارت دیگر اگر یک Bounded Context نتواند کار خود را به درستی انجام دهد تمام Bounded Context ها تحت تاثیر قرار خواهند گرفت.

تعریف Context Mapping:

تا به اینجا به تعریف Bounded Context و نوع ارتباطی که یک یا چند Bounded Context می‌توانند با یکدیگر داشته باشند پرداختیم. هر تیم نرم افزاری بر روی یک یا تعداد محدودی از Bounded Context ها کار می‌کند و اعضای تیم فقط به همان Context ها مسلط هستند و از داخل Context های دیگر اطلاعات زیادی ندارند.

از آنجایی که در طول عمر پروژه نرم افزاری، Bounded Context ها توسعه پیدا می‌کنند و Model ها دستخوش تغییر می‌شود و جزییات جدید به آنها اضافه می‌شود، ممکن است گاهی جزییاتی به Model یک Bounded Context اضافه شود که آن جزییات متعلق به Bounded Context دیگر باشد و با اضافه کردن آن جزییات به Bounded Context فعلی باعث می‌شود یک مفهوم در دو جای مختلف بازتعریف شوند و چه بسا که در جای غیراصلی خود به صورت کامل تعریف نشده باشد.

با وجود آنکه هر عضوی از تیم بر Bounded Context یی که متعلق به تیم است تسلط دارد، اما ممکن است به نوع ارتباطاتی که این Bounded Context با Bounded Context های دیگر دارد آگاه نباشد. بخشی از کار تیم های نرم افزاری توسعه سرویس مورد نیاز تیم های دیگر و یا ارائه اطلاعات مورد نیاز آن ها است. اینکه نوع ارتباط بین Bounded Context تیم شما با Bounded Context تیم دیگر به چه شکلی است قطعا بر توسعه سرویس ها و ارائه اطلاعات تاثیر مستقیم دارد. اگر توسعه دهنده از نوع ارتباط باخبر نباشد ممکن است سرویسی که توسعه می‌دهد توسط تیم دیگر قابل استفاده نباشد یا در صورت استفاده کیفیت قابل قبولی نداشته باشد و باعث به وجود آمدن چالش هایی بین دو تیم نرم افزاری شود.

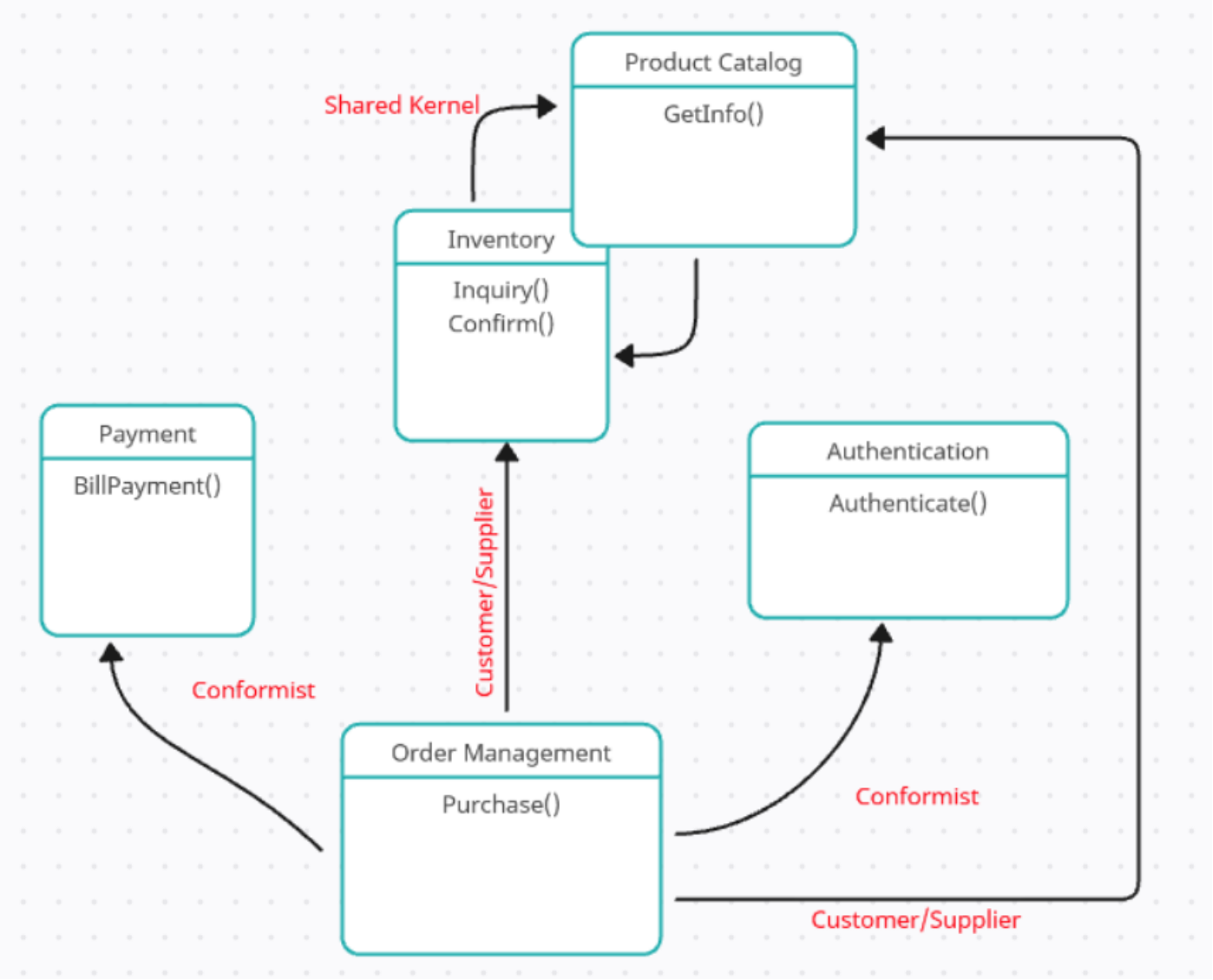
نتیجه ای که در نهایت این دو مشکل برای پروژه نرم افزاری به وجود می‌آورند، از بین بردن مرز بین Context ها است و از طرفی همانطور که در قسمت های قبلی گفته شد، مشخص کردن درست و دقیق مرز بین Context ها یکی از مهم ترین کارهایی است که در طراحی معماری یک نرم افزار انجام می‌شود و می‌تواند یکی از عوامل موفقیت یا شکست هر پروژه نرم افزاری باشد.

بنابراین اگر شما مرز بین Context ها را حتی درست تشخیص داده باشید و ترسیم کرده باشید در صورت عدم آگاهی اعضای تیم از Context های دیگر و یا عدم آگاهی از نوع ارتباط بین Bounded Context ها می‌تواند منجر به از بین رفتن همان مرز ها و در نتیجه شکست پروژه نرم افزاری شود.

برای جلوگیری از این مشکل باید یک نقشه از تمام Bounded Context ها تهیه کنید تا مشخص کنید، هر کدام روی کدام قسمت از Domain کار می‌کنند و هر کدام چه نوع ارتباطی با دیگر Bounded Context ها دارد. به این نقشه Context Map گفته می‌شود.

Context Map می‌تواند به شکل های مختلف تهیه شود و هر کدام برای منظور خاصی مورد استفاده قرار گیرد. برای مثال می‌توان یک Context Map از کل سیستم تهیه کرد و از آن برای جلسات با سهامداران و ذینفعان استفاده کرد یا برای آموزش نیروهای تازه وارد به منظور آشنایی با پروژه ها استفاده کرد. در این نوع Context Map معمولا به جزییات داخل هر Bounded Context پرداخته نمی‌شود. به عنوان مثال دیگر می‌توان شرایطی را در نظر گرفت که سه تیم نرم افزاری می‌خواهند یک فرآیندی را توسعه دهند که نیاز به برقراری ارتباط بین Bounded Context های خود به منظور رد و بدل کردن دیتا و ارائه سرویس دارند، در این شرایط یک Context Map کلی کارآمد نیست و بهتر است یک Context Map از Bounded Context های درگیر همراه با جزییات داخل هر Bounded Context تهیه کرد. این جزییات می‌تواند شامل Entity های موجود در هر Bounded Context و ارتباط بین آن ها باشد.

نمونه یک Context Map:



Context Map یک مستند بسیار مفید از طراحی سیستم نرم افزاری است که در طول عمر نرم افزار همراه با توسعه Model و باید مداوم بروزرسانی شود. وجود یک Context Map بروز در هر پروژه نرم افزاری باعث می‌شود تا تیم ها بدانند کدام Bounded Context با کدام Bounded Context دیگر ارتباط دارد و چه نوع ارتباطی دارد و هر کدام داخل خود چه Entity هایی با چه رفتار هایی دارند که مزایای زیر را به همراه دارد:

- تعامل بین تیم ها سریع تر و راحت پیش می‌رود چون هر تیمی می‌داند که چه تیمی از چه تیم دیگری باید تبعیت کند یا همکاری مشترک داشته باشند و یا ...
- معمار نرم افزار می‌تواند تصمیمات معماری قابل اطمینان تر اتخاذ کند
- تیم ها دیگر نیاز ندارند بسیاری از مسائلی که قبلا درمورد آنها تصمیم گیری شده است تحلیل یا بحث کنند چون Context Map تمام این موارد را داخل خود دارد

تقطیر سازی دامنه: در مرحله پنجم از طراحی استراتژیک به دنبال شناخت بهتر از دامنه کسب و کار هستیم که Eric Evans آن را به تقطیر سازی دامنه تشبیه می‌کند. در ادامه به بررسی روش هایی می‌پردازیم که به توسعه دهندگان پروژه کمک می‌کنند تا شناخت بهتری نسبت به دامنه کسب و کار داشته باشند و در نهایت بتوانند سیستم نرم افزاری پیاده سازی کنند که کسب و کار را به هدف اصلی برساند. این روش ها شامل این موارد هستند:

- سند چشم انداز دامنه
- مستند سازی دامنه
- جداسازی دامنه
- مکانیزم ها
- ریفکتورینگ
- Event Storming
- Domain Story Telling
- Example Mapping

سند چشم انداز دامنه: در شروع پروژه هیچ Model سازی انجام نشده و معمولا توسعه پروژه با یک Model ابتدایی شروع می‌شود. اما معمولا پروژه های نرم افزاری به مرور زمان بزرگ و پیچیده می‌شوند. در یک پروژه بزرگ و پیچیده افراد به راحتی دچار درسردرگمی می‌شوند و مسیر توسعه را اشتباه پیش می‌برند بدین ترتیب که ممکن است تمرکز خود را به جای اینکه بر روی مزیت رقابتی Domain صرف کنند، بر روی جزییات به ظاهر مهم اما در اصل بی اهمیت هزینه کنند. تمام ضینفعان پروژه از ابتدای مسیر باید بدانند پروژه نرم افزاری برای چه هدفی ایجاد شده و قرار است چه ارزش افزوده ای را به سازمان اضافه کند. با دانستن این مورد است که تمام افراد در طول عمر پروژه نرم افزاری می‌دانند بر کدام قسمت های مدل کسب و کاری باید تمرکز بیشتری داشته باشند و بیشترین تلاش را در آن قسمت انجام دهند. برای این منظور می‌توان، سند چشم انداز دامنه (Domain Vision Statement) تعریف کرد.

سند چشم انداز دامنه یک توضیح کوتاه درباره دامنه کسب و کار است که به صورت خلاصه ویژگی های کلیدی نرم افزار و مزیت های رقابتی که نرم افزار برای سازمان ایجاد می‌کند را توضیح می‌دهد. این سند برای تمام ضینفعان پروژه مفید است، به طوری که مدیران سازمان با استفاده از آن می‌دانند که منابع را به چه صورتی باید تقسیم کنند و یا در مدل سازی می‌دانند که برای کدام قسمت باید بیشترین تلاش را اعمال کنند و یا می‌توانند از آن برای آموزش نیروی جدید استفاده کنند.

سند چشم انداز دامنه یکی از تکنیک های تقطیر سازی دامنه است که کمک می‌کند مدل سازی موثر تر داشته باشید و پروژه در مسیر درست خود به پیشرفت ادامه دهد.

Eric Evans توصیه می‌کند این سند را خیلی خلاصه در حد یک صفحه بنویسید و در آن درباره Core Domain و ارزش افزوده ای که به سازمان اضافه می‌کند توضیح دهید. از توضیح دادن درباره جزییات و هر آن چیزی که خارج از Core Domain است بپرهیزید. توضیح دهید چگونه Core Domain به نیاز های مختلف پاسخ می‌دهد. از همان ابتدا با شروع پروژه این سند را تهیه کنید و همچنان که در طول پروژه دید عمیق تری نسبت به دامنه کسب و کار پیدا می‌کنید این سند نیز باید بروز شود.

مستند سازی دامنه: شما می‌توانید فرآیند تقطیر دامنه را به خوبی انجام دهید، انواع Subdomain های پروژه را شناسایی کنید، Bounded Context ها و مرز بین آنها را به درستی تشخیص دهید و مدل سازی کنید و خیلی خوب به Core Domain مسلط باشید، اما آیا پروژه نرم افزاری توسعه داده شده هم به خوبی تمام این موارد را نشان می‌دهد؟

نتیجه فرآیند تقطیر سازی چندین Subdomain است که هر کدام شامل چندین Bounded Context می‌شوند هر Bounded Context شامل چند Module و هر Module شامل چندین Component است. پس تعداد اجزای تشکیل دهنده بسیار زیاد هستند. یکی از Sub Domain های شناسایی شده در فرآیند تقطیر، Core Subdomain است، که اگر در پروژه توسعه داده شده قابل تشخیص نباشد به راحتی بین آن همه اجزای تشکیل دهنده پروژه نرم افزاری دفن و غیر قابل شناسایی می‌شود. بنابراین در پروژه نرم افزاری در حال توسعه باید طوری Core Domain را پیاده سازی کنید تا از باقی Subdomain ها قابل تشخیص باشد. اگر توسعه دهندگان در پروژه نرم افزاری قادر به تشخیص Core Domain نباشند در واقع قادر به تشخیص نیستند که چه بخشی در پروژه از اهمیت بیشتری برخوردار است و باید بیشترین تلاش خود را در کدام قسمت ها بگذارند. در نتیجه در چنین شرایطی توسعه دهندگان به همان میزان زمانی که صرف توسعه Generic Subdomain می‌کنند به همان میزان برای Core Domain هزینه میکنند. خب قاعدتا اهمیت Core Domain بسیار بیشتر از Generic Domain است پس اگر برای هر دو بخش به یک اندازه هزینه کنید یعنی در حال هدر دادن سرمایه های پروژه هستید و به احتمال زیاد این پروژه نرم افزاری با شکست مواجه خواهد شد.

Eric Evans برای جلوگیری از این مشکل تکنیک Highlighted Core را معرفی کرده است. او می‌گوید در هنگام پیاده سازی و توسعه پروژه ساختار پروژه را طوری طراحی و پیاده سازی کنید که Core Domain از باقی قسمت ها به راحتی قابل تشخیص باشد به طوری که وقتی یک توسعه دهنده جدید به تیم اضافه می‌شود با نگاه کردن به پروژه به راحتی بتواند تشخیص دهد Core Domain کدام قسمت پروژه قرار دارد. حتی اگر با یک پروژه Legacy مواجه هستید که Core Domain قابل تشخیص نیست می‌توانید با ابزار هایی که در محیط توسعه در اختیار دارید Core Domain را قابل تشخیص کنید مثلا می‌توانید در قسمت هایی که Core Domain پیاده سازی شده است با استفاده از Comment نوشتن برای دیگران مشخص کنید که این قسمت مربوط به Core Domain است. پس تا می‌توانید در طراحی ساختار پروژه و نام گذاری ها سعی کنید Core Domain از باقی قسمت ها به راحتی قابل تشخیص باشد.

طراحی درست ساختار پروژه و نام گذاری درست در توسعه پروژه یکی از روش های مستند کردن Core Domain است که در محیط کدنویسی قابل پیاده سازی است. روش دیگر که مکمل این روش است مستند کردن Core Domain به واسطه دیاگرام ها است. Eric Evans توصیه می‌کند یک سند بین 3 تا 7 صفحه از Core Domain تهیه کنید و تمام اجزای Core Domain به همراه تمام ارتباطاتی که Core Domain با باقی اِلمان های پروژه دارد را داخل آن سند توضیح بدهید یا با ترسیم دیاگرام آن را نمایش بدهید. این دیاگرام ها می‌توانند دیاگرام های استانداردی مثل UML باشند یا حتی یک دیاگرام غیر استاندارد که روی کاغذ نرسیم شده باشند. در واقع تنها چیزی که در این سند مهم است این است که Core Domain و تمام ارتباطات آن با تمام اجزای پروژه به خوبی مشخص شده باشد. اگر در طول عمر پروژه Core Domain تغییر کرد این سند نیز باید تغییر کند و تغییر در آن باید با تایید مدیران پروژه باشد و بعد از تغییر باید فورا نسخه جدید سند در اختیار تمام اعضای تمام تیم ها قرار بگیرد و به تمام آنها اطلاع رسانی لازم انجام شود.

جداسازی دامنه: مستند سازی دامنه بر روی نحوه نام گذاری Core Domain در پروژه تاکیید دارد به نحوی که از اسم هر اِلمان بتوان تشخیص داد که آن بخشی از Core Domain است. اما نام گذاری درست تا حدودی می‌تواند از گم شدن Core Domain در میان انبوهی از کد محافظت کند. در راستای تقطیر دامنه یک تکنیک دیگری که می‌تواند این محافظت را تقویت کند، جدا کردن Core Domain در پروژه از باقی قسمت ها به صورت ساختاری است.

در هر پروژه نرم افزاری بخش قابل توجهی از فایل ها و کدهای موجود مربوط به رابط کاربری، زیرساخت های مورد نیاز، کدهای پشتیبانی و عمومی هستند که اگر Core Domain هم داخل آنها قرار گیرد به راحتی در میان آن حجم از کد گم می‌شود. بنابراین Core Domain باید در پروژه شما جایگاه مخصوص به خود را داشته باشد تا آن را از باقی قسمت ها به صورت فیزیکی جدا کند.

با نام گذاری درست و ساختاربندی صحیح Core Domain همیشه در پروژه مشخص است که قلب تپنده پروژه کجا قرار گرفته و چه ویژگی هایی دارد و در واقع کسب و کار با استفاده از آن پروژه کدام یک از نیاز مشتریان خود را پاسخ می‌دهد.

مکانیزم ها در Sub Domain ها: هر Domainیی یک سری مکانیزم ها برای انجام یک سری کار ها دارد. این مکانیزم ها در داخل خود یک سری الگوریتم دارند و یا مجموعه ای از چند فرآیند پیچیده هستند تا یک هدفی را به سرانجام برسانند. برای مثال مکانیزم خواندن پیام از صف را در نظر بگیرید. این مکانیزم در دل خود یک توالی از کارها را انجام می‌دهد تا به یک سرور وصل شود سپس روی آن سرور به یک تکنولوژی خاصی که زیرساخت صف را فراهم می‌کند وصل می‌شود و با پروتکلی که آن تکنولوژی معرفی کرده از صف پیام ها را دریافت کند. حال که Domain برای اهداف خود نیاز به این مکانیزم دارد آیا باید این مکانیزم در دل Doamin پیاده سازی شود؟ یا به عبارت دیگر آیا Domain باید از نحوه انجام عملیات این مکانیزم با خبر باشد؟ قطعا خیر.

هر Domain مربوط به مسئله خاص است و هر آن چیز مهمی که به آن مسئله می‌پردازد و شخصیت آن مسئله را شکل می‌دهد داخل Domain مربوطه قرار می‌گیرد و هر آن چیزی که در شخصیت Domain نقشی ندارد خارج از آن قرار می‌گیرد. پس زمانی که Domain می‌خواهد از این مکانیزم استفاده کند نباید بداند که این مکانیزم چگونه کار خود را انجام می‌دهد، بلکه فقط باید بداند چه کاری انجام می‌دهد. در واقع مکانیزم ها باید خارج از Domain پیاده سازی شوند و دسترسی Domain به آنها فقط از طریق Interface خواهد بود. هر Interface می‌تواند شامل چندین مکانیزم باشد که تمام آنها در یک زمینه خاص سرویس می‌دهند مثلا در زمینه صف ها می‌توان یک Interface با دو مکانیزم خواندن از صف و نوشتن در صف داشت. همچنین اسم هر مکانیزم باید بیانگر کاری که آن سرویس انجام می‌دهد باشد و هیچ اشاره ای به اینکه چگونه آن کار را انجام می‌دهد نباید داشته باشد.

از آنجایی که Interface مکانیزم برای استفاده Domain نوشته می‌شود، باید این Interface را داخل Domain نگهداری کنیم. به عبارت دیگر Domain مالک این Interface است. اما از آنجایی که Domain نباید از نحوه انجام کار آن Interface نباید اطلاعی داشته باشد و نباید به آن وابسته باشد پس باید پیاده سازی آن Interface را خارج از Domain انجام دهیم.

آیا می‌توان مکانیزم ها را از Core Subdomain به Generic Subdomain ها منتقل کرد؟ خیر. Core Subdomain و Generic Subdomain هر دو یک دامنه دارند و از لحاظ ساختار و اجزای تشکیل دهنده شبیه یکدیگر هستند، تنها تفاوت آن ها در میزان اهمیت آنها است. Core Subdomain از اهمیت بسیار بیشتری برای کسب

و کار برخوردار است در حالی که اهمیت Generic Subdomain بسیار پایین تر است. بنابراین خود Generic Subdomain ها شامل یک سری مکانیزم ها هستند که باید خارج از Domain آن پیاده سازی شود. پس فارغ از آنکه مکانیزم داخل Core Subdomain استفاده می‌شود یا Generic Subdomian باید داخل همان Subdomain از طریق یک Interface سرویس دهی کند و Domain هیچ گاه از چگونگی انجام کار آن نباید مطلع شود.

یک استثنا وجود دارد که یک مکانیزم می‌تواند داخل خود domain پیاده سازی شود. اگر خود مکانیزم جزو مزیت رقابتی کسب و کار باشد، آنگاه باید داخل خود Domainپیاده سازی شود.

Domain Story Telling: در هر کسب و کاری دامنه های زیادی وجود دارد که انواع مختلفی نیز دارند مثل Core و Support و Generic و هر کدام تقسیم به چند Bounded Context می‌شوند که در هر کدام یک سری موجودیت وجود دارد. برخی از موجودیت ها نقش دارند و یک یا چند کار مشخص را با ترتیب مشخصی انجام می‌دهند و برخی دیگر از موجودیت ها اشیایی هستند که کارها روی آنها انجام می‌شوند.

دانش مربوط به موجودیت ها و اشیا و سناریو های ممکن در اختیار یک نفر نیست. معمولا این دانش بین تمام ضینفعان پروژه مشترک است. یعنی بخشی از این دانش در اختیار مدیران کسب و کار بخشی دیگر در اختیار تیم محصول و ... است.

از آنجایی که توسعه دهنده سیستم قرار است پروژه نرم افزاری را پیاده سازی کند، باید این دانش را در اختیار داشته باشد. یک تکنیک بسیار مفید که جزو طراحی استراتژیک محسوب می‌شود تکنیک Domain Story Telling است. معمولا از این تکنیک در مراحل ابتدایی و قبل شروع پروژه استفاده می‌شود تا به کمک آن بتوانند Subdomain ها، Bounded Contextها، موجودیت ها و محدودیت ها را زودتر شناسایی کنند و در نهایت Model سازی صحیح تری از کسب و کار داشته باشند.

در این تکنیک تیم محصول و تیم توسعه و در صورت نیاز مدیران مربوطه یک جلسه تشکیل می‌دهند و بر روی یک یا چند فرآیند کسب و کار با یکدیگر گفتوگو می‌کنند. در طی این گفتوگو دانش مربوط به آن فرآیندها از افراد به یکدیگر منتقل می‌شود و این دانش از طریق دیاگرام هایی که روی تخته ترسیم می‌شود به شکل ظاهری در می‌آید. این دیاگرام ها، دیاگرام های رسمی مثل UML یا دیاگرام های تکنولوژیک نیستند بلکه یه نوع زبان تصویری هستند برای نمایش آن چه که در Domain توسط افراد روی اشیا اتفاق می‌افتد. در این زبان تصویری چهار اِلمان طراحی وجود دارد که به واسطه آنها باید فرآیند مورد نظر را ترسیم کنید، که عبارتند از:

- بازیگر (Actor)
- شی (Work Object)
- عمل (Activity)
- ترتیب (Sequence Number)

بازیگر و شی موجودیت های Domain هستند و عمل، رفتارهایی است که متعلق به بازیگر است و یا رخدادی است که برای یک شی اتفاق می‌افتد و ترتیب مشخص می‌کند که رفتارها با چه ترتیبی توسط بازیگر اجرا می‌شوند و یا رخدادها با چه ترتیبی شکل می‌گیرند. شما راجع به هر فرآیندی این 4 اِلمان را مشخص کنید توانسته اید دانش خود را نسبت به دامنه تقویت کنید.

فرآیند های مورد نظر با سناریو های ممکن توسط این چهار اِلمان ترسیم می‌شوند. توجه داشته باشید هر چقدر در فرآیند ها به جزییات بیشتری بپردازید تعداد سناریو ها بیشتر و در نتیجه یا تعداد دیاگرام ها بیشتر

می‌شود یا دیاگرام های بزرگتری ترسیم می‌شود و این موضوع ممکن است انتقال دانش مورد را نظر را سخت کند.

تمام افرادی که در این جلسه شرکت می‌کنند بعد از جلسه دانش عمیق تری نسبت به دامنه خواهند داشت. آنها می‌دانند موجودیت هایی که در این قسمت از کسب و کار دخیل هستند کدام ها هستند، اولویت هر کدام نسبت به دیگری چگونه است. در حین این گفتوگو زبان مشترک تقویت می‌شود و اصطلاحات بیشتری به آن اضافه می‌شود و یا اصطلاحات قبلی معنای شفاف تری برای افراد پیدا می‌کنند. در نتیجه وقتی تمام افراد درگیر پروژه دانش عمیق تری نسبت به کسب و کار داشته باشند باعث می‌شود نوع و تعداد دامنه ها دقیق تر مشخص شود و همچنین باعث می‌شود تعداد Bounded Context و مرز بین آنها با ضریب خطای کمتری مشخص شود.

تفاوت Domain Vision State و Domain Story Telling: سند Domain Vision State ماموریت یک دامنه را مشخص می‌کند. شما در این سند اهمیت دامنه مورد نظر برای کسب و کار را توضیح می‌دهید در حالی که Domian Story Telling جزییات فرآیندهای موجود در یک Subdomain را مورد بررسی قرار می‌دهد. در واقع شما با استفاده از Domain Story Telling می‌توانید ماموریت دامنه را شناسایی کنید و سپس به کمک آن سند Domain Vision State را تولید کنید.

Event Storming: یکی دیگر از راه های شناخت بهتر Domain شناخت رخداد های ممکن در دامنه است. یعنی تمام اتفاقاتی که در یک Subdomain رخ می‌دهد به همراه عامل آن ها و توالی آنها را شناسایی کنید. به این تکنیک Event Storing گفته می‌شود. همانند تکنیک Domain Story Telling نیاز دارید تا تمام کسانی که در مورد آن Domain دانش دارند در یک جلسه دور هم جمع کنید و در مورد رخدادهای آن Domain با یکدیگر گفتوگو و بحث کنید یا به عبارت دیگر یک طوفان فکری درباره تمام رخدادهای ممکن راه بیاندازید.

اِلمان های تشکیل دهنده:

Event: یک اتفاقی که در دامنه رخ می‌دهد

Command: عاملی که یک Event را شکل می‌دهد

Aggregate: عاملی که Event را مدیریت می‌کند

Policy: دستورالعمل بعد از رخداد Event

Hotspot: چیزهایی که در فرآیند Event Storming برایمان نامشخص هستند

برای مدیریت بهتر این اِلمان ها بهتر است از برگه های کوچک رنگی بر روی تخته استفاده کنید. معمولا Event ها را با رنگ زرد، Command ها با رنگ نارنجی، Aggregate ها با رنگ سبز، Policy ها با رنگ آبی و Hotspot ها با رنگ قرمز مشخص می‌شوند. همچنین برای این کار ابزارهای آنلاینی وجود دارند که این کار رو برای شما آسان تر می‌کنند.

برای شروع ابتدا تمام Event های ممکن را شناسایی کنید و طبق یک خط زمانی از چپ به راست آنها را بچینید، زمان وقوع Event ها همیشه در گذشته است پس تمام Event باید فعل گذشته داشته باشند، سپس از انجایی که هر Event توسط یک Command به وجود می‌آید، شروع به شناسایی Command هر Event کنید. Command هر Event را پشت سر آن قرار دهید. زمان هر Command باید زمان حال باشد. سپس شروع به شناسایی Aggregate ها کنید. تمام Event ها و Command ها یک موضوعیتی دارند مثلا سفارش، مشتری. برای مثال بهتر فرض کنید یک Command داریم با عنوان "سفارش ثبت می‌شود" و یک Event برای آن داریم با عنوان "سفارش ثبت شد"، موضوعیت این دو "سفارش" هستند پس سفارش، Aggregate مربوط به این دو می‌باشد. پس از شناسایی Aggregateها آنها را میان Command ها و Event ها قرار دهید. قاعدتا بعد از هر Event باید یک Command دیگر اجرا شود تا فرآیند به مرحله بعدی برود اینکه مرحله بعدی چه چیزی باید باشد توسط Policy مشخص می‌شود و این Policy خود یک Command دیگر را اجرا می‌کند. و در نهایت چیزهایی که برای شما نامشخص هستند را به صورت Hotspot در نظر بگیرید تا در جلسات بعدی بتوانید با تبادل دانش بیشتر آنها را حل کنید.

در انتهای این جلسه شما موفق به شناسایی تمام Event ها، Command ها، Aggregate ها و Policy های موجود در یک Domain شده اید و با دانش عمیقی که به دست آورده اید می‌توانید Subdomain ها، Bounded Context ها، مرز و نحوه ارتباط بین Bounded Context ها را صحیح تر شناسایی کنید.

تفاوت Event Storming و Domain Story Telling: تکنیک Domain Story Telling بر روی فرآیندها و آدم هایی که آن فرآیندها را انجام می‌دهند تمرکز دارد در حالی تکنیک Event Storming خیلی جزیی تر دامنه را هدف قرار می‌دهد و تمرکز اصلی آن روی رخداد ها است.به عبارت دیگر می‌توان گفت Domain Story Telling حالت داستان سرایی دارد و بیشتر روی بخش توضیح کسب و کاری یک Domain تمرکز دارد و کمک می‌کند آن

Domain را بهتر درک کنید اما Event Storming بیشتر جنبه فنی دارد و کمک می‌کند تا پیاده سازی صحیح تری از Domain داشته باشید.

Example Mapping: یکی دیگر از تکنیک هایی که کمک به شناخت بهتر دامنه و پیاده سازی صحیح آن می‌کند Example Mapping است. شما به کمک این تکنیک می‌توانید قوانین موجود در دامنه را شناسایی کنید و با نوشتن مثال های متنوع آن قوانین را به راحتی درک کنید. این تکنیک جزو تکنیک های معرفی شده در DDD نیست اما استفاده از آن می‌تواند در شناخت دامنه کمک قابل توجهی کند.

برای شروع استفاده از این تکنیک نیاز به شناسایی User Story های دامنه دارید. اما User Story چیست؟

User Story تعریف کوتاهی است از کاری که یک کاربر می‌خواهد توسط قابلیت هایی که در کسب و کار ما وجود دارد انجام دهد و در قبال آن یک ارزشی برای آن کاربر ایجاد می‌شود. هر User Story یک انجام دهنده کار دارد که یک نقشی را در کسب و کار ایفا می‌کند مثلا مشتری، مدیر کسب و کار، مسئول انبار و ... ، یک کار مشخصی دارد که با انجام شدن آن یک ارزش مشخصی برای کاربر انجام دهنده ایجاد می‌شود.

برای استفاده از این تکنیک افراد تیم در یک اتاق کنار هم قرار جمع می‌شوند که حتما در افراد حاضر باید حداقل یک تحلیلگر، یک توسعه دهنده و یک تستر وجود داشته باشد حضور باقی افراد در صورت نیاز بالامانع است. این تکنیک 4 اِلمان دارد و هر کدام با یک رنگ مشخص می‌شوند:

- User Stroy: رنگ زرد
- قوانین: رنگ سبز
- مثال: رنگ آبی
- سوال یا ابهام: رنگ قرمز

ابتدا User Story را روی یک کارت زرد می‌نویسید سپس آن را بر روی تخته قرار می‌دهید. حال تمام قوانین مربوط به آن رو روی کارت های سبز رنگ می‌نویسید و زیر کارت زرد قرار می‌دهید. این قوانین همان هایی هستند که اگر هر کدام از آنها رعایت نشود User Story انجام نمی‌شود. برای هر قانون یک یا چند مثال روی کارت های آبی می‌نویسید تا به خوبی مساله را درک کنید و آن ها زیر کارت های سبز قرار می‌دهید. در این حین اگر در مورد هر قسمت ابهامی وجود دارد که هیچکس پاسخی برای آن ندارد می‌توانید آن را بر روی کارت قرمزی بنویسید و در زیر کارتهای آبی قرار دهید.

مدت جلسه برای هر User Story بهتر است نهایتا 30 دقیقه باشد. اگر در 30 دقیقه جلسه نتیجه ای نداشت می‌توانید جلسه دیگری برنامه ریزی کنید و تا آن جلسه دیگر پاسخ کارت های قرمز را نیز پیدا کنید. اگر تعداد قوانین زیاد شد و یا تعداد سناریو های مختلف برای مثال ها زیاد شد تا جایی که یک جلسه برای آن کافی نبود بهتر است در صورت امکان آن User Story را به چند User Story دیگر تقسیم کنید.

در اتمام هر جلسه افراد حاضر در جلسه نسبت به دامنه دید عمیق تری پیدا کرده اند و همچنین نیروهای توسعه دهنده بهتر می‌توانند برای پیاده سازی این User Story زمان بندی دقیق تری ارائه دهند.

یکی از خروجی های مهم این جلسه این است که مشخص می‌شود برای عملکرد صحیح دامنه چه تست هایی باید توسط توسعه دهنده نوشته شود و این مورد کمک زیادی به پیاده سازی صحیح دامنه می‌کند.

ارتباط Domain Story Telling و Event Storming و Example Mapping:

هر سه تکنیک برای شناخت بهتر دامنه استفاده می‌شوند اما شما می‌توانید به انتخاب خود از هر کدام که تشخیص دادید استفاده کنید. در واقع اجباری برای استفاده از هر سه تکنیک در DDD وجود ندارد. این نوع تصمیمات بستگی به شرایط پروژه و کسب و کار دارد که معمار سیستم با در نظر گرفتن این شرایط در مورد انتخاب تکنیک های استراتژیک تصمیم گیری می‌کند.

اگر بودجه و زمان کافی دارید، استفاده از هر سه تکنیک به صورت جداگانه، کمک زیادی به شناخت بهتر دامنه می‌کند.

با استفاده از Domain Story Telling فرآیندها و هدف هر دامنه را شناسایی کنید.

با استفاده از Event Storming رخداد ها، دستورات مربوط به ایجاد رخداد ها و سیاست های دامنه ها را شناسایی کنید.

با استفاده از Example Mapping ارزش هایی که هر دامنه برای کاربران مختلف با رعایت قوانین کسب و کاری به ارمغان می‌آورد را شناسایی کنید.

Core Domain Rafactoring

تکنیک هایی که تا به اینجا مورد بررسی قرار دادیم کمک به پیاده سازی بهتر Core Domain و حفظ انسجام آن در پروژه می‌کند. حال اگر با پروژه ای روبرو شدید که از این تکنیک ها برای پیاده سازی Core Domain استفاده نشده بود و با یک Core Domain ضعیف روبرو بودید باید با استفاده از این تکنیک ها سعی کنید تا انسجام را به آن بازگردانید و نسخه بهتری از Core Domain را در پروژه پیاده سازی کنید.

برای شروع ریفکتورینگ پروژه بهتر است از یکی از دو مورد زیر شروع کنید:

- اگر پروژه با مشکلات جدی روبرو است، به دنبال ریشه مشکلات باشید. اگر ریشه مشکل در Core Domain یا در ارتباطات بین Core Domain با باقی قسمت ها بود، برای حل مشکل، ریفکتورینگ را از همین نقطه شروع کنید.
- اگر مشکل جدی گریبان گیر پروژه نیست و می‌توانید ریفکتورینگ را از هر قسمت شروع کنید، بهتر است سراغ Core Domain بروید و سعی کنید با تکنیک جداسازی دامنه نسخه بهتری از آن را پیاده سازی کنید یا به سراغ Supporting Subdomain هایی بروید که می‌توانند به عنوان Generic Subdomain در پروژه پیاده سازی شوند.

فرآیند طراحی فنی

پس از یاد گرفتن طراحی استراتژی و تکنیک های آن می‌توانید دامنه و موضوعات حول محور آن را به خوبی شناسایی کنید، شما می‌توانید با کمک این تکنیک ها فضای بزرگ و پیچیده مسئله را به مسائل کوچک تر با شفافیت بیشتر تقسیم کنید و سپس برای هر کدام از آنها راه حل خود را ارائه دهید. در نهایت تمام این راه حل ها در کنار هم، معماری و طراحی سیستم نرم افزاری را شکل می‌دهند که باعث می‌شود Core Domain برای اهدافی که به وجود آمده خدمت رسانی کند و در نهایت باعث حل مشکلات کاربران سیستم و درآمدزایی برای کسب و کار شود.

حال برای پیاده سازی راه حل ها باید طراحی فنی بلد باشید تا بتوانید راه حل های استراتژیک خود را به درستی پیاده سازی کنید.

هر پروژه نرم افزاری یک ساختاری برای پیاده سازی دارد. ساختار پروژه ها می‌تواند به قسمت های مختلف تقسیم شود. هر قسمت را یک لایه می‌توان در نظر گرفت. ساده ترین نوع ساختار، ساختار یک لایه است. در این نوع ساختار کل پیاده سازی کنار هم قرار می‌گیرد. بخشی از پروژه نرم افزاری را کدهای مربوط به رابط کاربری نرم افزار در بر می‌گیرد. در واقع این رابط کاربری همان ظاهر نرم افزار است جایی که استفاده کنندگان نرم افزار از آن برای تعامل با نرم افزار استفاده می‌کنند. بخشی دیگر از پروژه مربوط به زیرساخت تکنولوژی های مورد استفاده در نرم افزار است که به توسعه دهنده قابلیت ذخیره سازی دیتا ، ارسال پیام و پردازش اطلاعات را می‌دهد. بخش دیگر مربوط به domain است جایی که قلب تپنده نرم افزار است و دلیل وجود نرم افزار.

در ساختار یک لایه تمام کدهای مربوط به زیرساخت، رابط کاربری و دامنه در یک لایه قرار می‌گیرند. طبق تجربه ثابت شده اگر دامنه بزرگ و پیچیده باشد بعد از مدتی تمام این بخش ها با یکدیگر وابستگی شدید پیدا می‌کنند به طوری که کد ها در قسمت هایی توسعه پیدا می‌کنند که به متعلق به آنجا نیستند. مثلا کدهای مربوط به دامنه در بین کدهای مربوط به رابط کاربری نوشته می‌شوند. در این نوع پروژه ها بعد از مدتی نگهداری پروژه غیر ممکن می‌شود چراکه در صورت تغییر در بخشی از نرم افزار ممکن است بخش دیگر دچار مشکل شود یا برای تغییر یک قانون خاص دامنه نیاز به تغییرات گسترده در قسمت های مختلف پروژه باشد. اما به طور کل ساختار یک لایه همیشه بد نیست، در واقع به علت اینکه ساختار یک لایه به صورت کوتاه مدت سرعت توسعه را بالا می‌برد و هزینه را کاهش می‌دهد می‌توان، از آن برای پروژه های بسیار کوچک یا پروژه های موقت، استفاده کرد. برای مثال اگر می‌خواهید یک دامنه کوچک با قابلیت های خیلی کم پیاده سازی کنید می‌توانید از ساختار یک لایه استفاده کنید. یا اگر قصد پیاده سازی یک نمونه اولیه از یک نرم افزار برای دمو دارید می‌توانید از ساختار یک لایه استفاده کنید. اما برای پروژه با دامنه های بزرگ باید چه کاری کرد؟

راه حلی که مهندسی نرم افزار طی چند سال اخیر به سمت آن سوق پیدا کرده ساختار چند لایه است.
شماتیک کلی این ساختار از 4 لایه تشکیل شده است:

- User Interface
- Application
- Domain
- Infrastructure

در لایه User Interface تمام کدهای مربوط ظاهر نرم افزار نوشته می‌شود که راه ورود کاربران به سیستم است.

در لایه Infrastructure تمام کدهای مربوط به زیرساخت پروژه برای استفاده از تکنولوژی های مورد نیاز نوشته می‌شود. مثل کدهای مربوط به برقرار ارتباط به دیتابیس و ذخیره سازی داده یا کدهای مربوط به برقرار ارتباط با سیستم های دیگر. هر گونه برقراری ارتباط با دنیایی بیرون از طریق این لایه انجام می‌شود.

در لایه Domain کدهای مربوط به کسب و کار و قوانین آن و راه حل هایی که برای مشتریان خود ارائه می‌دهد نوشته می‌شود. این لایه باید کاملا ایزوله باشد، به این معنی که به هیچ کدام از لایه ها نباید وابستگی داشته باشد و هر کاری که مربوط به خدمات اصلی کسب و کار است باید داخل این لایه بدون وابستگی به لایه های دیگر پیاده سازی شود. عدم وابستگی این لایه، به توسعه دهنده این امکان را می‌دهد که بتواند تست های مهم مربوط به صحت سنجی عملکرد دامنه را نیز بدون نیاز به لایه های دیگر و راحت تر بنویسد.

لایه Application شبیه به یک مدیر عمل می‌کند. هیچ درخواستی مستقیم از لایه User Interface به سمت لایه Domain ارسال نمی‌شود بلکه به سمت لایه Application ارسال می‌شود سپس این لایه تصمیم می‌گیرد که درخواست را به چه صورت به سمت لایه Domain ارسال کند. این لایه ترتیب توالی کار ها را مدیریت می‌کند و منطق مربوط به نحوه عملکرد نرم افزار در این لایه پیاده سازی می‌شود. اما هیچ کدام از منطق های کسب و کار در این لایه پیاده سازی نمی‌شود.

در این نوع لایه بندی، آن هایی اهمیت بیشتری دارند لایه های سطح بالا حساب می‌شوند و آن هایی که اهمیت کمتری دارند لایه های سطح پایین محسوب می‌شوند. توصیه می‌شود لایه های سطح پایین به لایه های سطح بالا وابسته باشند نه برعکس. پس طبق این توصیه، لایه User Interface به لایه Application وابسته است. لایه Application به لایه Domain وابسته است. لایه Infrastructure به لایه Domain و لایه Application وابسته است ولی لایه Domain به هیچ کدام از لایه ها وابسته نیست.

تمام اجزایی که داخل لایه Domain قرار می‌گیرند نباید دخالتی در نمایش دادن خود در رابط کاربری، ذخیره سازی خود در لایه Infrastructure ، مدیریت توالی کارها در لایه Application داشته باشند، بلکه تمام تمرکز آنها باید بر نشان دادن کسب و کار و قوانین آن باید باشد. با این روش دامنه قابلیت تکامل دارد و می‌تواند هر لحظه دانش کسب و کار را بیشتر وارد پروژه کند و موفقیت پروژه را رقم بزند.

نکته دیگری که باید در مورد وابستگی لایه ها اعمال کنید، قانون وابستگی ضعیف است. طبق این قانون اول سعی کنید وابستگی هایی که ایجاد می‌کنید یک طرف باشد و نه دو طرفه و دوم سعی کنید تا جای ممکن به پیاده سازی ها وابستگی نداشته باشید بلکه به انتزاعی که پیاده سازی ها آز آنها پیروی می‌کنند وابسته باشید.

برای مثال فرض کنید لایه Infrastructure یک سری خدمات برای ذخیره سازی دیتا داخل دیتابیس ارائه می‌کند و این خدمات را از طریق یک سری سرویس که داخل کلاس SomethingRepository پیاده سازی شده ارائه می‌کند. حال توسعه دهنده نیاز دارد این سرویس ها را داخل لایه Application فراخوانی کند، اگر این سرویس ها بدون انتزاع پیاده سازی شده باشند، توسعه دهنده مجبور است از این پیاده سازی به صورت مستقیم استفاده کند و این یعنی وابستگی قوی به لایه Infrastructure و به پیاده سازی خدمات مربوط به دیتابیس، ولی اگر برای آن پیاده سازی یک انتزاع داشته باشد مثلا ISomethingRepository ، توسعه دهنده به جای استفاده از پیاده سازی (SomethingRepository) از انتزاع آن پیاده سازی استفاده می‌کند (ISomethingRepository) و با این کار وابستگی به پیاده سازی خدمات مربوط به دیتابیس از بین رفته ولی همچنان از آنجایی که آن انتزاع داخل لایه Infrastructure قرار گرفته، همچنان وابستگی Application به

Infrastructure وجود دارد، لذا اگر آن انتزاع (ISomethingRepository) را داخل خود لایه Application قرار دهیم، از آنجایی لایه Infrastructure به لایه Application وابسته است می‌تواند به انتزاع دسترسی داشته باشد و آن را پیاده سازی کنید از طرفی لایه Application دیگر به لایه Infrastructre وابستگی محکمی ندارد و از طرفی از خدمات Infrastructure نیز می‌تواند استفاده کند.

معمار نرم افزار در شروع پروژه با توجه به تصمیماتی که در طراحی استراتژی گرفته، ساختار پروژه و زیرساخت های مورد نیاز و تکنولوژی های مورد نیاز را در پروژه انتخاب می‌کند سپس تیم توسعه آن را پیاده سازی می‌کند. همچنان که پروژه رو به جلو حرکت می‌کند ممکن است تغییراتی در ساختار پروژه و زیرساخت ها و تکنولوژی های مورد استفاده اعمال شود که تمام این تغییرات در راستای بهبود پروژه و هدف کسب و کار خواهند بود. در ادامه به بررسی روش هایی می‌پردازیم که به توسعه دهنده سیستم کمک می‌کند تا پیاده سازی بهینه تری از domain در پروژه داشته باشد.

فرآیند پیاده سازی Model در نرم افزار

بعد از مدل سازی کردن domain نوبت به پیاده سازی آن در پروژه می‌رسد. به طور کلی و خلاصه Model به دست آمده تشکیل شده از یک سری اِلمان است که یک شبکه ارتباطی بین آنها برقرار است و یک سری فرآیند در کسب و کار را پیش می‌برند. حال DDD برای پیاده سازی این اِلمان ها سه الگو معرفی کرده است:

- Entity
- Value Object
- Service

در ادامه به بررسی هر کدام می‌پردازیم.

Entity: در هر کسب و کاری یک سری موجودیت وجود دارد که خدمت رسانی کسب و کار وابسته به وجود این موجودیت هاست. این موجودیت می‌تواند یک شی باشد مثلا شی ماشین یا می‌تواند یک موجود جاندار باشد مثلا مشتری. هر موجودیت یک سری ویژگی ها و یک سری رفتار ها دارد. کسب و کار با استفاده از ویژگی ها و رفتارهای این موجودیت ها به کاربران خود خدمات ارائه می‌کند. هر موجودیت ویژگی ها و رفتارهای بیشماری می‌تواند داشته باشد، اما، تنها آن ویژگی ها و رفتارهایی را در پروژه پیاده سازی می‌کنیم که برای کسب و کار اهمیت دارد.

تمام Entity ها دو ویژگی خاص دارند که هنگام پیاده سازی باید رعایت شوند. این دو ویژگی عبارتند از:

- شناسه منحصر به فرد
- تداوم در طول عمر خود

هر Entity باید یک شناسه منحصر به فرد در کل کسب و کار داشته باشد تا به واسطه همین شناسه شناسایی شود. همچنین یک Entity در طول عمر خود ممکن است بارها تغییر وضعیت دهد یا برخی از ویژگی هایش بروزرسانی شوند که در این صورت هرگز نباید شناسه آن تغییر کند. به عبارت دیگر شناسه هر Entity هویت آن موجودیت است از طرفی ویژگی های آن Entity ممکن است تغییر کند اما هویت آن همیشه ثابت است.

برای مثال در یک فروشگاه اینترنتی، موجودیتی به اسم مشتری وجود دارد که برای پیاده سازی آن از الگوی Entity باید استفاده کرد چرا که هر مشتری داخل این فروشگاه اینترنتی باید یک هویت منحصر به فرد داشته باشد. پس احتمالا توسعه دهنده این فروشگاه اینترنتی یک کلاس ایجاد می‌کند به اسم Customer و تمام ویژگی ها و رفتارهای مشتری را داخل این کلاس قرار می‌دهد و احتمالا برای شناسه آن یک فیلد به اسم Id

قرار می‌دهد و با استفاده از منابعی که دارد برای هر مشتری یک شناسه منحصر به فرد تولید می‌کند و داخل این فیلد Id قرار می‌دهد.

هر Entity در دنیای واقعی یک سری رفتارها و وظایف و قوانین مشخص دارد، مثلا یک مشتری در یک فروشگاه آنلاین رفتارها، وظایف و قوانین زیر را دارد:

- تایید کردن ایمیل
- تغییر آدرس منزل
- اضافه کردن محصول به سبد خرید
- فعال کردن حساب کاربری
- افزایش امتیاز حساب کاربری
- وارد کردن اطلاعات شخصی

تمام این رفتارها، وظایف و قوانین باید در داخل کلاس Customer پیاده سازی شود چون متعلق به خود موجودیت است. به این نوع طراحی، طراحی رفتار محور گفته می‌شود. در واقع به جای تمرکز بر داده ها بر رفتارها تمرکز می‌کند چرا که هدف از طراحی به سبک DDD، شناخت چگونه کار کردن سیستم است پس در هنگام مدل سازی کردن کسب و کار علاوه بر شناسایی ویژگی های مهم هر موجودیت که همان داده ها هستند، بر رفتارهای موجودیت ها نیز تمرکز می‌کنیم، در نهایت Entity که ساخته می‌شود و در کد پیاده سازی می‌شود ترکیبی از ویژگی ها، رفتارها، وظایف و قوانین مربوط به آن Entity است.

Value Object: دسته دیگری از اِلمان های Model هستند که هویت آنها اهمیت ندارد و تنها مقداری که داخل خود نگهداری می‌کنند برایمان مهم است. به این المان ها Value Object گفته می‌شود. ویژگی های هر Entity نوعی از Value Object ها هستند. مثلا فیلد آدرس در موجودیت مشتری یک نوع Value Object حساب می‌شود. حال این Value Object می‌تواند یک رشته کاراکتر ساده باشد یا می‌تواند یک Class در دنیای برنامه نویسی باشد که درون خود چند Value Object دیگر نگهداری می‌کند. در دنیای برنامه نویسی شی گرا ساده ترین نوع Value Object ها نوع داده ها هستند، یعنی به فرض مثال نوع داده String یا Int یک نوعی از Value Object ها به حساب می‌آیند. در مثالی که برای آدرس در موجودیت مشتری گفتیم، آدرس می‌تواند یک نوعی از String در کلاس Customer باشد یا می‌تواند یک کلاس به اسم Address باشد که درون خود شهر و استان و محله را به عنوان سه نوع داده String نگه می‌دارد.

به عبارت دیگر درباره Entity ها "کیستی" و "هویت" آن برای ما مهم است اما درباره Value Object ها "چیستی" و "مقدار" آن برای ما مهم است.

اگر دو Entity با شناسه های متفاوت تمام ویژگی هایشان با هم یکسان باشند نمی‌توان گفت آن دو Entity با هم برابر هستند، چون هویت آنها با هم متفاوت است اما اگر دو Value Object مقدار یکسانی داشته باشند می‌توان گفت آن دو با یکدیگر برابر هستند، چون هویت ندارند و با مقدارشان شناسایی می‌شوند، پس زمانی که مقدار یکسان داشته باشند باهم برابرند. بنابراین Value Object ها باید به صورتی پیاده سازی شوند که بعد از متولد شدن دیگر مقادیر آنها تغییر نکنند، چرا که در صورت تغییر اعتبار خود را از دست داده اند.

همانند Entity ها Value Object ها نیز می‌توانند رفتار، وظایف و قوانین مربوط به خود را داشته باشند که در این صورت تمام این موارد باید در کلاسی که برای آن Value Object می‌نویسیم پیاده سازی کنیم.

Service: یک سری فرآیندهایی که در کسب و کار باید انجام شوند جزو رفتارها، وظایف و قوانینی هستند که می‌توان مسئولیت آنها را به Entity ها و Value Object ها واگذار کرد، اما یک سری از فرآیندها هستند که نمی‌توان مسئولیت آنها را به هیچ Entity یا Value Object یی واگذار کرد، این فرآیندها باید به صورت مستقل پیاده سازی شوند.

اگر سعی کنید این فرآیندها را در یکی از Entity ها بگنجانید، اتفاقی که می‌افتد اینست که آن Entity مفهوم خود را در پروژه از دست می‌دهد و بعد از مدتی دچار پیچیدگی های نارمرتبط با خود می‌شود. هر Entity با توجه به ویژگی ها و رفتارهایی که برای آن در مدل سازی شناسایی می‌شود یک مرز مفهومی در دامنه کسب و کار برای خود تعریف می‌کند، با وارد کردن وظایف نا مرتبط به داخل آن Entity باعث از بین رفتن مرزهای آن Entity می‌شوید. بعد از مدتی وقتی افراد تیم به آن Entity نگاه می‌کنند دلیل وجود این Entity را متوجه نخواهند شد و مفهوم آن را درک نخواهند کرد حتی اگر تصمیم بگیرند آن را ریفکتور کنند به علت وابستگی های نامرتبط و وابسته شدن اشیا دیگر پروژه به آن Entity به واسطه آن فرآیند، باعث می‌شود Refactor پروژه کاری سخت و پرهزینه باشد.

الگوی سوم برای پیاده سازی Model الگوی Service است. طبق این الگو هر فرآیندی که جزوی از رفتار ها، وظایف و قوانین هیچ Entity و Value Object یی نباشد باید به صورت یک Service مستقل ایجاد شود. متدی که داخل کلاس Service پیاده سازی می‌شود معمولا چند ورودی می‌گیرد و فرآیند مورد را نظر را روی آنها اعمال می‌کند سپس در خروجی در اختیار استفاده کنندگان قرار می‌دهد. Service ها همیشه بدون نگهداری وضعیت هستند یعنی سرویس فراخوانی می‌شود، کار خود را انجام می‌دهد، خروجی را به بیرون پاس می‌دهد و هیچ دیتایی داخل خود نگهداری نمی‌کند، پس اگر هزاران Object سرویس را در لحظه فراخوانی کنند سرویس از هیچ کدام هیچ دیتایی نگهداری نمی‌کند و بعد از اجرا شدن اطلاعی از دیتا هایی در ورودی دریافت کرده بود یا در خروجی به بیرون پاس داده بود، نخواهد داشت. کلاسی که برای پیاده سازی Service در نظر می‌گیرید بهتر است داخل اسم خود از افعال استفاده کرده باشد چرا که آن سرویس برای یک فرآیند مشخص به وجود آمده و بهتر است برای واضح تر بودن وظیفه Service کاری که انجام می‌دهد همراه با فعل کار داخل اسم کلاس Service باشد.

برای مثال فرآیند جابه جایی پول بین جساب های بانکی در نرم افزار بانکی را درنظر بگیرید. از آنجایی که برای این کار نیاز به دو Account Entity هست پس ایده خوبی نیست که این فرآیند داخل خود Entity پیاده سازی کنیم چرا که در آن صورت داخل خود Entity باید یک شی دیگر از خود Entity نگهداری کنیم پس بهتر است یک کلاس بسازیم به اسم TransferMoneyService و سپس داخل این کلاس یک متد بسازیم به اسم Transfer که در ورودی دو Object از جنس Account دریافت می‌کند و پول را از یکی به دیگری انتقال می‌دهد.

Modules: بعد از شناسایی Entity ها ، Value Object ها و Service های موجود در دامنه باید آنها در جای مناسبی در پروژه قرار داد. قبل تر در مورد اینکه دامنه را چگونه به صورت فیزیکی در پروژه قرار بدهیم صحبت کردیم و پیشنهاد شد برای اینکه دامنه در پروژه مشهود باشد بهتر است یک پوشه به اسم Domain در پروژه قرار دهیم و هر آن چیزی که مربوط به دامنه است داخل این پوشه قرار بگیرد.

Entity ، Value Object و Domain Service ها مواردی هستند که باید در پوشه Domain قرار بگیرند، اما، نحوه قرار گیری آنها مهم است. زمانی که فقط یک Entity در دامنه مربوطه وجود داشته باشد می‌توان آن یک مورد را به تنهایی داخل پوشه Domain قرار داد اما زمانی که تعداد زیادی Entity ، Value Object و Domian Service داشته باشیم و هر کدام به یک قسمتی از دامنه مربوط باشند بهتر است آنها را دسته بندی کنیم به طوری که اسم هر دسته بندی گویای داستان آن دسته بندی باشد. به هر کدام از این دسته بندی ها یک Module گفته

می‌شود. فرض کنید در دامنه یک پروژه این موجودیت ها را پیاده سازی کرده اید:
Customer.cs ، CustomerRegisteredEvent.cs ، CustomerService.cs ، Order.cs ، OrderService.cs ، در این سناریو می‌توانید دو Module بسازید یکی به اسم Customer و دیگری به اسم Order و موارد مربوطه را داخل هر کدام قرار دهید.

```
Domain
├─ Customers
│   ├─ Customer.cs
│   ├─ CustomerRegisteredEvent.cs
│   └─ CustomerService.cs
│
└─ Orders
    ├─ Order.cs
    └─ OrderService.cs
```

با ایجاد این دسته بندی برای توسعه دهندگان بعدی پروژه یک نقشه ذهنی می‌سازید که هر کسی که به این ساختار نگاه کند متوجه می‌شود داخل دامنه چه مفاهیمی و چه روابطی وجود دارد و جزییات هر کدام چیست. به عبارت دیگر توسعه دهندگان هر پروژه با بررسی Module های موجود در پروژه می‌توانند دو موضوع بسیار مهم را دامنه یاد بگیرند:

- نوع رابطه بین Module ها و مفاهیم و مسائل سطح بالا در دامنه
- جزییاتی که داخل هر کدام از مفاهیم دامنه وجود دارد و برای تحقق اهداف دامنه ضروری هستند

Aggregates: در هر دامنه ای چندین Entity و Value Object وجود دارد. بسیاری از آنها با یکدیگر ارتباط دارند، از طرفی برای انجام یک سری فرآیند ها بیش از یک Entity و Value Object درگیر می‌شوند. مشکلی که ممکن است در این فرآیندها پیش بیآید عدم یکپارچگی و در نهایت نقض صحت عملکرد سیستم است.

فرض کنید دو Entity دارید به اسم های Entity A و Entity B که در یک فرآیندی این دو باید با یکدیگر تغییر کنند و از طرفی یک قانونی در دامنه دارید به اسم Rule A که بعد برای تغییر هر کدام از آنها باید این قانون رعایت شود، در این شرایط شما مجبور هستید در طول انجام فرآیند به صورت یک تراکنش، تغییرات خود را اعمال کنید تا مطمئن شوید Entity A و Entity B با یکدیگر تغییر کرده اند و داخل تراکنش Rule A را چک کنید، از طرفی در تمام فرآیندهایی که هر کدام از Entity های A و B تغییر می‌کنند باید Rule A را چک کنید تا مطمئن شوید قوانین دامنه نقض نشده باشد. کنترل کردن چنین شرایطی بدین شکل، مخصوصا در سیستم های بزرگ کاری بسیار پیچیده به حساب می‌آید.

راه حلی که DDD برای این شرایط در نظر گرفته، استفاده از Aggregate ها است. در واقع در هر دامنه Entity هایی که با یکدیگر ارتباط دارند و داخل یک مرز مشخصی از لحاظ مفهوم قرار می‌گیرند می‌توان داخل یک گروه مشخصی قرار داد. به هر کدام از این گروه ها Aggregate گفته می‌شود.

برای ساخت یک Aggregate یک کلاس ایجاد می‌کنید و تمام Entity ها و Value Object هایی که در این گروه قرار گرفته اند را داخل Aggregate قرار می‌دهید. از این پس برای تغییر هر کدام از Entity ها، Aggregate یک تراکنش باز می‌کند و هر Entity که لازم باشد داخل این تراکنش تغییرات لازم را می‌پذیرد و بعد یا قبل از

انجام تغییرات تمام قوانین دامنه را یک بار چک می‌کند تا از صحت عملکرد فرآیندها مطئمن شود. هر Aggregate یک Aggregate Root دارد که یکی از Entity های موجود است و ارتباط دنیای بیرون با این Aggregate از طریق همین Aggregate Root خواهد بود. قوانینی که توسط Aggregate ها باید رعایت شوند:

- دسترسی دنیای بیرون به Aggregate فقط از طریق Aggregate Root ممکن است
- دنیای بیرون به Entity های موجود در Aggregate دسترسی ندارد مگر در مواقع لازم که آن هم به صورت موقت و محدود است
- شناسه Aggregate Root دارای شناسه عمومی است که از دنیای بیرون برای همه قابل دسترسی است ولی شناسه Entity های دیگر داخلی است و فقط از داخل Aggregate قابل دسترسی است
- از آنجایی که دسترسی به Entity های داخل Aggregate فقط از طریق Aggregate Root ممکن است، بنابراین فقط برای یافتن Aggregate Root می‌توان در دیتابیس Query زد و باقی Entity ها باید توسط Aggregate Root از دیتابیس واکشی شود
- تمام Entity های موجود در Aggregate برای دسترسی به Aggregate دیگر می‌توانند از Aggregate Root آن Aggregate این کار را انجام دهند
- هر تغییری که روی هر کدام از Entity های یک Aggregate انجام شود باید کل Aggregate توسط Aggregate Root صحت سنجی شود تا مطمئن شد هیچ کدام از قوانین نقض نشده است

بنابراین تمام Entity ها و Value Object هایی که با یکدیگر ارتباط دارند و یک سری قوانین کسب و کار باید با استفاده از تمام آنها رعایت شوند را در یک Aggregate قرار دهید. این Aggregate یک مرز فرضی حول تمام این Entity ها و Value Object ها محسوب می‌شود. با هر تغییری داخل این مرز یک بار تمام قوانین داخل مرز باید مجدد چک شوند تا صحت Aggregate تایید شود. هیچ شی ای از دنیای بیرون به داخل این مرز دسترسی ندارد مگر به Aggregate Root. در مواقع لازم دنیای بیرون می‌تواند به اعضای داخل این مرز دسترسی موقت و فقط خواندنی داشته باشد، به عبارت دیگر اگر یک شی ای از خارج Aggregate نیاز به یکی از اعضای داخل آن نیاز داشته باشد، آن شی را از Aggregate دریافت می‌کند اطلاعات مورد نیاز خود را استخراج می‌کند سپس هیچ رفرنسی به آن شی نگه نمی‌دارد و هیچ تغییر در آن ایجاد نمی‌کند. بدین ترتیب می‌توان تضمین کرد قوانین داخل Aggregate همیشه رعایت می‌شوند.

Factories: در دنیای واقعی مفهوم کارخانه را می‌توان بدین شکل تفسیر کرد: جایی که مواد اولیه را دریافت کرده و محصول نهایی را تولید می‌کند. در توسعه پروژه نرم افزاری شما نیاز به ایجاد اشیا مختلف از انواع مختلف دارید. بسیاری از این اشیا همان مفاهیمی هستند که در دامنه کسب و کار زندگی می‌کنند مثل مفهوم "سفارش" یا مفهوم "مشتری". ایجاد کردن یک شی از یک مفهوم ساده، اغلب اوقات کار راحتی است، اما، اگر بخواهید یک شی جدید از یک Aggregate ای که داخل خود چندین Entity نگهداری می‌کند ایجاد کنید، ممکن است کار راحتی نباشد، چرا که، برای ایجاد این شی جدید باید از تمام Entity های داخل آن Aggregate نیز یک شی جدید بسازید، از طرفی ممکن است برای ساخت تمام Entity های داخل Aggregate نیاز به یک سری پردازش ها داشته باشید، همه این موارد کنار هم باعث می‌شوند ساخت شی جدید از یک Aggregate کار نسبتا پیچیده ای شود. DDD برای ساخت این اشیا پیچیده استفاده از Factory را پیشنهاد می‌دهد. دقیقا مثل همان تعریف ابتدای مبحث، Factory یک کلاس یا که متد است که با دریافت تمام اطلاعات لازم در ورودی می‌تواند یک شی جدید از Aggregate مورد نظر برای Client تولید کند در حالی که تمام پیچیدگی های ایجاد آن شی داخل Factory کپسوله شده و تمام قوانین موجود در Aggregate رعایت شده.

چرا استفاده از Factory بهتر از استفاده از Constructor است؟ با ایجاد شی جدید از Entity ها داخل Constructor، در واقع آن Aggregate را به صورت شدید به Entity های داخل خود وابسته کرده اید و هر تغییری در آن Entity ها منجر به تغییر در Constructor آن Aggregate می‌شود که این رویکرد در تناقص با رویکرد وابستگی ضعیف در اصول مهندسی نرم افزار است. از طرفی اگر Client بخواهد از تمام Entity های داخل Aggregate، یک شی جدید بسازید در واقع نسبت به جزییات داخل Aggregate آگاهی پیدا می‌کند و از طرفی باید برای رعایت قوانین Aggregate آن قوانین را سمت خود چک کند که تمام این موارد یعنی نقض قانون کپسوله سازی.

Factory را می‌توان به دو روش Factroy Method و Abstract Factory پیاده سازی کرد. در روش اول یک متد داخل Aggregate پیاده سازی می‌کنید که وظیفه آن دریافت اطلاعات لازم و ساخت یک شی جدید از خودش است. در روش دوم یک کلاس می‌سازید و متدهای Factory را در آن پیاده سازی می‌کنید. چه زمانی از کدام روش استفاده کنیم؟ زمانی که شما فقط می‌خواهید از یک Aggregate یک شی جدید بسازید بهتر است Factory Method را داخل همان Aggregate پیاده سازی کنید ولی زمانی که نیاز دارید از چند Aggregate یک شی جدید به صورت یک جا بسازید باید از روش دوم استفاده کنید.

در هر دو روش برای رعایت وابستگی ضعیف توصیه می‌شود برای Aggregate خود را از یک Interface ارث بری کنید و خروجی Factory نوعی از آن Interface باشد.

هنگام ساخت شی جدید در Factory، وظیفه صحت سنجی قوانین Aggregate به عهده Aggregate Root و صحت سنجی قوانین داخلی هر Entity به عهده خود Entity است. حتما برای حالتی که، دیتای ورودی، قوانین Aggregate را نقض می‌کند، یک استاندارد برای خروجی در نظر بگیرید مثلا می‌توانید در چنین شرایطی Exception برگردانید یا دیتای null برگردانید.

Repository: یکی از مهم ترن قسمت های هر نرم افزار، بخش نگهداری دیتا است. معمار نرم افزار با توجه به دغدغه ها و پیش نیاز های کسب و کار یک یا چند تکنولوژی برای نگهداری دیتای نرم افزار انتخاب می‌کند. از طرفی هر نرم افزاری دیتا های زیادی اعم از رخدادها، تاریخچه ها و اطلاعات مورد نیاز کاربران و کسب و کار را در دیتابیس ذخیره و نگهداری می‌کند. نکته ای که در اینجا حائز اهمیت است، نحوه دسترسی به دیتابیس برای ذخیره سازی و خواندن دیتا از دیتابیس می‌باشد. در طول عمر نرم افزار، در فرآیند های گوناگون، نرم افزار نیاز دارد با دیتابیس ارتباط برقرار کند، اما، نحوه برقراری این ارتباط بسیار مهم است. اگر بخواهید در همه جای نرم افزار مستقیما به دیتابیس دسترسی پیدا کنید و دیتا را واکشی کنید یا آن را ذخیره کنید سه مشکل بزرگ در پروژه شما به وجود خواهد آمد:

- وابستگی شدید تمام قسمت های نرم افزار به تکنولوژی دیتابیس مربوطه
- نقض قوانین دامنه به واسطه پخش شدن قوانین دامنه در پروژه
- تغییر رویکرد معماری از Domain-Centric به Database-Centric

فرض کنید قصد دارید به مشتریانی که تا کنون بیش از 1000 دلار از شما خرید کرده اند یک پیامک کد تخفیف ارسال کنید. احتمالا در این سناریو در لایه Presentation یک ایجنت دارید که یک سرویس از لایه Application فراخوانی می‌کند، اگر شما در لایه Application بخواهید مستقیما به دیتابیس وصل بشوید و لیست این مشتریان را استخراج کنید در واقع به صورت مستقیم این لایه را به تکنولوژی که برای اتصال به دیتابیس استفاده می‌کنید وابسته کرده اید. از طرفی مشخص کردن این موضوع که کدام مشتری شامل این

تخفیف می‌شود یک قانون مهم کسب و کاری است و باید توسط لایه Domain تعیین و تکلیف شود، پس عملا مرزبندی Domain را از بین برده اید و مرکزیت نرم افزار را از Domain-Model-Centric که هدف اصلی DDD است به Database-Model-Centric تغییر داده اید.

برای جلوگیری از این اتفاق، DDD الگوی Repository را برای برقرار ارتباط با دیتابیس پیشنهاد می‌دهد. Repository را می‌توان به پلی میان نرم افزار و دیتابیس تشبیه کرد که تمام کوئری ها و کدهای مربوط به تکنولوژی دیتابیس در آنجا پیاده سازی می‌شود و هر قسمتی از نرم افزار که نیاز داشته باشد هر دیتایی را از دیتابیس بخواند یا در دیتابیس ذخیره کند فقط از طریق Repository این کار را انجام خواهد داد.

دو قانون مهم در پیاده سازی Repository:

- هر Repository باید یک Interface داشته باشد
- فقط Aggregate ها می‌توانند Repository داشته باشند

Repository دو کار مهم انجام می‌دهد، یکی تعامل با دیتابیس از طریق تکنولوژی مخصوص آن دیتابیس و دیگری جابه جایی دیتا بین دیتابیس و نرم افزار. اگر Repository ها را بدون Interface تعریف کنید تمام قسمت هایی که با Repository ها سر و کار دارند به آنها وابستگی پیدا می‌کنند و اگر روزی طراح نرم افزار تصمیم بگیرد تکنولوژی دیتابیس پروژه را تغییر دهد احتمال دارد علاوه بر تغییر Repository ها نیاز به تغییر در استفاده کنندگان Repository ها نیز داشته باشید. پس برای هر Repository یک Interface داخل لایه Domain تعریف کنید و پیاده سازی آن را داخل لایه Infrastructure انجام دهید و هر زمانی که نیاز به تغییر تکنولوژی داشته باشید بدون نیاز به تغییر در جاهای مختلف، فقط Repository را تغییر خواهید داد.

از آنجایی که تمام قوانین کسب و کار از طریق Aggregate ها پیاده سازی می‌شوند و برای ایجاد هر کدام از Entity های داخل آن نیاز به کنترل کردن قوانین دامنه توسط آن Aggregate است، پس برای آنکه اجازه ندهیم Entity های داخل هر Aggregate بدون دخالت Aggregate ایجاد شوند و از نقض قوانین دامنه جلوگیری کنیم، هیچ کدام از Entity ها نباید Repository داشته باشند. در واقع از آنجایی که Entity ها از طریق Aggregate ها ایجاد می‌شوند، برای واکشی دیتای آنها از دیتابیس یا ذخیره سازی آنها در دیتابیس باید از طریق Repository مربوط به آن Aggregate این کارها را انجام داد. پس زمانی که می‌خواهید یک Aggeragte را از طریق Repository از دیتابیس واکشی کنید باید تمام Entity های داخل آن را نیز از واکشی کنید تا Aggregate بتواند تمام قوانین را یک بار کنترل کند.

به یاد داشته باشید در Aggregate ها، مدل و قوانین کسب و کار را پیاده سازی می‌کنید، اما، لزوما دقیقا همان مدل را داخل دیتابیس خود نخواهید داشت مثلا ممکن است در Customer Aggregate یک لیستی از آدرس های مشتری داشته باشید اما در دیتابیس یک جدول به نام Customer خواهید داشت و یک جدول به نام CustomerAddress. در واقع می‌توان گفت Domain Model قوانین کسب و کار را پیاده سازی می‌کند و Database Model فقط نحوه ذخیره سازی دیتا های مورد نیاز کسب و کار را. توجه داشته باشید برای حفظ یکپارچگی نرم افزار تا جای ممکن این دو مدل را شبیه به یکدیگر نگهداری کنید اما هرگز نباید عینا مثل هم باشند.

جمع بندی

مقاله ای که شما مطالعه کردید، درک من از رویکرد DDD از مطالعه کتاب Domain Driven Design Tackling Complexity in the Heart of Software نوشته Eric Evans همچنین مطالعه مقالات مربوط به مباحث DDD در وبسایت های مختلف بود. چیزی که من از مطالعه این منابع یاد گرفتم این بود که DDD واقعا تنها یک روش برای پیاده سازی پروژه نرم افزاری نیست، بلکه، یک نوع نگرش به کسب و کار و تولید نرم افزار برای آن است، بدین معنی که اگر شما این نگرش را در خود ایجاد کنید، متوجه می‌شوید، الزامی برای استفاده از تمام DDD در پروژه های نرم افزاری وجود ندارد بلکه می‌توانید فقط بخش هایی از آن را استفاده کنید که نیاز شما را در پروژه برطرف می‌کند.

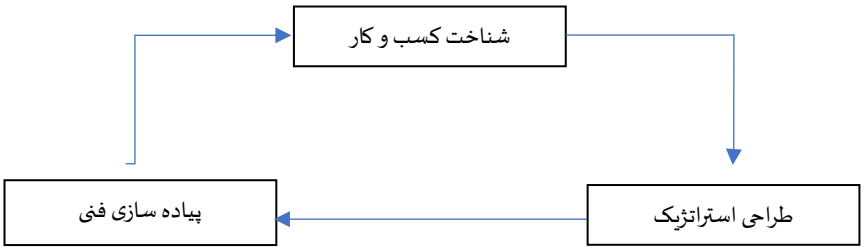
من در مطالعات خود یاد گرفتم برای راه اندازی یک نرم افزار با رویکرد DDD در ابتدا نیاز به مطالعه کسب و کار و شناخت آن است. سپس با استفاده از روش هایی که در طراحی استراتژیک معرفی شده است باید اقدام به شناسایی Sub Domain ها کرد، سپس با توجه به شرایط کسب و کار، معمار نرم افزار به همراه صاحبان کسب و کار تصمیم می‌گیرند کدام Sub Domain ها توسط تیم توسعه پیاده سازی شود، کدام برونسپاری شود و برای کدام از محصولات آماده استفاده شود. در مرحله بعد برای Sub Domain هایی که تصمیم به توسعه آنها توسط تیم توسعه کسب و کار گرفته شده است، شروع به شناسایی Bounded Context های آنها می‌کنیم. در این مرحله میتوانیم برای شناسایی بهتر مرز بین Sub Domain ها و Bounded Context ها از روش هایی مثل Domain Story Telling، Event Storming و Example Mapping استفاده کنیم.

بعد از مدل سازی کسب و کار با استفاده از روش های طراحی استراتژیک نوبت به پیاده سازی به فنی می‌رسد. باز هم چیزی که من یاد گرفتم این است که به ازای هر Bounded Context یک پروژه داریم که با توجه به لایه بندی هایی که برای پروژه در نظر می‌گیریم، اجزای Bounded Context را در آن لایه بندی های پیاده سازی می‌کنیم. هر لایه بندی به اقتضای نیاز های پروژه می‌تواند از یک تا چند Component داشته باشد و هر Component بخشی از نیاز های Bounded Context را برطرف می‌کند. نام یکی از این Component ها Domain است که داخل آن Entity ها، Value Object ها، Domain Service ها و قراردادهایی که Domain به آنها برای ارتباط با باقی لایه ها نیاز دارد، قرار می‌گیرند. و از باقی لایه ها برای خدمت رسانی به Domain یا ارتباط با دنیای بیرون استفاده می‌شود.

و آخرین نکته ای که می‌خواهم به آن اشاره کنم: فرآیند شناخت کسب و کار سپس طراحی استراتژیک و سپس پیاده سازی فنی، یک فرآیند خطی نیست، یعنی نقطه شروع دارد ولی، پایان ندارد در عوض یک فرآیند دورانی است که باید در طول عمر پروژه بارها و بارها تکرار شود چرا که در شروع پروژه هر چقدر هم از کسب و کار شناخت داشته باشیم باز هم ناشناخته های زیادی برای ما در کسب و کار وجود خواهد داشت یا در طول عمر کسب و کار نیاز های جدید اضافه می‌شوند و یا نیاز های قبلی تغییر می‌کنند. حال فرض کنید دانش جدیدی نسبت به کسب و کار به دست آورده اید یا نیازمندی جدیدی از سمت کسب و کار درخواست داده شده است و یا نیازمندی های قبلی نیاز به تغییر دارند، تمام این موارد باعث تغییر در مدل سازی ای می‌شود که ما از کسب و کار در طراحی استراتژیک داشته ایم. وقتی مدل سازی قبلی دچار تغییر شود به ناچار باید پیاده سازی فنی هم تغییر کند تا مدل و پیاده سازی همگام با یکدیگر پیش بروند. و این نکته، نکته ای هست که Eric Evans مداوم به آن اشاره کرده است، او بار ها و بار ها در کتاب خود یادآوری می‌کند چرخه توسعه پروژه باید به گونه ای باشد که افراد تیم به صورت مداوم با یکدیگر در ارتباط باشند تا بتوانند دانش خود از دامنه کسب و کار را عمیق تر کنند، سپس باید مدل قبلی را اصلاح کنند و از طرفی پیاده سازی همیشه

باید نمایشی صریح از مدل طراحی شده باشد و بین مدل و پیاده سازی نباید تفاوتی وجود داشته باشد، پس، به محض تغییر در مدل شروع به ریفکتور کد کنید به طوری کد نمایانگر مدل باشد.

در نهایت می‌توانید دیاگرام زیر را به عنوان مسیر توسعه به روش DDD در ذهن خود به یاد داشته باشید.



ممنون از اینکه این مقاله را مطالعه کردید.

مهدی کاظمی

https://www.linkedin.com/in/mahdi-kazemi-career

منابع

Domain-Driven Design: Tackling Complexity in the Heart of Software
https://hackolade.com/downloads/DDDM-eBook.pdf
https://deviq.com/domain-driven-design/shared-kernel/
https://mehmetozkaya.medium.com/shared-kernel-pattern-in-domain-driven-design-ddd-21cba2a9f92a
https://ddd-practitioners.com/home/glossary/bounded-context/bounded-context-relationship/shared-kernel/
https://medium.com/@dangeabunea/integrating-bounded-context-for-ddd-beginners-63c21af875fb
https://ddd-practitioners.com/home/glossary/bounded-context/bounded-context-relationship/conformist/
https://ddd-practitioners.com/home/glossary/bounded-context/bounded-context-relationship/anticorruption-layer/
https://ddd-practitioners.com/home/glossary/bounded-context/bounded-context-relationship/separate-ways/
https://ddd-practitioners.com/home/glossary/bounded-context/bounded-context-relationship/open-host-service/
https://buildsimple.substack.com/p/strategic-ddd-the-challenging-life
http://ddd.fed.wiki.org/open-host-service.html
https://ddd-practitioners.com/home/glossary/bounded-context/bounded-context-relationship/partnership/
https://buildsimple.substack.com/p/strategic-ddd-the-art-of-partnership
https://ddd-practitioners.com/home/glossary/bounded-context/bounded-context-relationship/
https://ddd-practitioners.com/home/glossary/bounded-context/
https://github.com/ddd-crew/context-mapping
https://www.infoq.com/articles/ddd-contextmapping/
https://deviq.com/domain-driven-design/subdomain/
https://ddd-practitioners.com/2023/03/07/the-difference-between-domains-subdomains-and-bounded-contexts/
https://medium.com/nick-tune-tech-strategy-blog/domains-subdomain-problem-solution-space-in-ddd-clearly-defined-e0b49c7b586c
https://medium.com/@lambrych/domain-driven-design-ddd-strategic-design-explained-55e10b7ecc0f
https://vaadin.com/blog/ddd-part-1-strategic-domain-driven-design
https://deviq.com/domain-driven-design/strategic-design/
https://socadk.github.io/design-practice-repository/activities/DPR-StrategicDDD.html
https://medium.com/@monge.gaspard.1746/strategic-patterns-of-ddd-domain-driven-design-6e4ae7dbf136
https://ddd.academy/strategic-ddd-using-bounded-context-canvas-gien-verschatse/
https://opus.ch/ddd-concepts-and-patterns-distillation-of-the-core-domain/
https://roluquec.medium.com/building-better-software-the-role-of-domain-storytelling-in-ddd-95091a93d89b
https://ddd-practitioners.com/home/glossary/domain-storytelling/
https://www.lucidchart.com/blog/ddd-event-storming
https://ddd.academy/event-storming-example-mapping/
https://www.kranio.io/en/blog/de-bueno-a-excelente-en-ddd-comprender-los-patrones-de-repositorios-en-domain-driven-design---4-10
https://www.abrahamberg.com/blog/repository-pattern-in-ddd-bridging-the-domain-and-data-models/
https://yottahmd.medium.com/mastering-ddd-repository-design-patterns-in-go-2034486c82b3
https://dev.to/axeldlv/domain-driven-design-part-1-strategic-design-30b2
https://martinfowler.com/bliki/DomainDrivenDesign.html
https://medium.com/ssense-tech/domain-driven-design-effective-domain-modeling-and-its-perks-e4e3e3e0d5ee
https://yoan-thirion.medium.com/domain-driven-design-re-distilled-685e75595a60
https://www.oreilly.com/library/view/domain-driven-design-distilled/9780134434964/
https://medium.com/building-inventa/how-we-used-event-storming-meetings-for-enabling-software-domain-driven-design-401e5d708eb